This version did **not** undergo peer review.

Any additional change to the schedule or the proposal are not reported.

Please refer to the **Open Access** version of record published by Elsevier: DOI: 10.1016/j.ijhcs.2025.103648

arXiv:2502.02194v3 [cs.HC] 6 Oct 2025

Highlights

Understanding User Mental Models in AI-Driven Code Completion Tools: Insights from an Elicitation Study

Giuseppe Desolda, Andrea Esposito, Francesco Greco, Cesare Tucci, Paolo Buono, Antonio Piccinno

- Developers expressed diverse preferences on when and how code suggestions should be triggered; this highlights the need for both user control and context-sensitive automation.
- The position to visualize code suggestion and explanation can be inline for short completions; sidebar, pop-up, or chatbot for longer or more complex suggestions.
- Most users want to receive minimal code suggestions and then request details on demand, including support for varying from code statements to whole functions or files.
- Explanations should be concise, contextual, and user-triggered (via click or shortcut), with links to documentation or example code.
- Developers prefer to customize activation timing, visualization style, explanation detail, and code style according to their personal preferences and project needs.

Understanding User Mental Models in AI-Driven Code Completion Tools: Insights from an Elicitation Study

Giuseppe Desolda[a,*], Andrea Esposito[a], Francesco Greco[a], Cesare Tucci[a,b], Paolo Buono[a], Antonio Piccinno[a]

[a]*Department of Computer Science, University of Bari Aldo Moro, Via E. Orabona 4, Bari, 70125, Italy*

[b]*University of Salerno, Via Giovanni Paolo II 132, Fisciano (Salerno), 84084, Italy*

Abstract

Integrated Development Environments increasingly implement AI-powered code completion tools (CCTs), which promise to enhance developer efficiency, accuracy, and productivity. However, interaction challenges with CCTs persist, mainly due to mismatches between developers' mental models and the unpredictable behavior of AI-generated suggestions, which is an aspect underexplored in the literature. We conducted an elicitation study with 56 developers using co-design workshops to elicit their mental models when interacting with CCTs. Different important findings that might drive the interaction design with CCTs emerged. For example, developers expressed diverse preferences on when and how code suggestions should be triggered (proactive, manual, hybrid), where and how they are displayed (inline, sidebar, popup, chatbot), as well as the level of detail. It also emerged that developers need to be supported by customization of activation timing, display modality, suggestion granularity, and explanation content, to better fit the CCT to their preferences. To demonstrate the feasibility of these and the other guidelines that emerged during the study, we developed ATHENA, a proof-of-concept CCT that dynamically adapts to developers' coding preferences and environments, ensuring seamless integration into diverse workflows.

Keywords: Code Completion Tools, Generative AI, Mental Models

*Corresponding author

Email addresses: giuseppe.desolda@uniba.it (Giuseppe Desolda), andrea.esposito@uniba.it (Andrea Esposito), francesco.greco@uniba.it (Francesco Greco), ctucci@unisa.it (Cesare Tucci), paolo.buono@uniba.it (Paolo Buono), antonio.piccinno@uniba.it (Antonio Piccinno)

URL: https://orcid.org/0000-0001-9894-2116 (Giuseppe Desolda), https://orcid.org/0000-0002-9536-3087 (Andrea Esposito), https://orcid.org/0000-0003-2730-7697 (Francesco Greco), https://orcid.org/0000-0001-5181-7115 (Cesare Tucci), https://orcid.org/0000-0002-1421-3686 (Paolo Buono), https://orcid.org/0000-0003-1561-70736 (Antonio Piccinno)

1. Introduction

Integrated Development Environments (IDEs) support developers with valuable tools to write and debug code in current software development practices. In many IDEs, code suggestion by automated systems has become a key element. Employing ever-developing Artificial Intelligence (AI) capabilities, these *Code Completion Tools (CCTs)* intend to boost developer efficiency, accuracy, and productivity by cutting down on the manual input needed for repetitive code segments and speeding up the entire coding task (Weber et al., 2024; Cui et al., 2024; Peng et al., 2023). CCTs powered by AI, like GitHub Copilot and JetBrains AI, indicate a movement towards advanced and informed programming assistance.

When dealing with CCTs, while potentially benefitting from increased efficiency, users are exposed to a wide range of risks strictly related to automation. For example, one of the main risks in the domain of code completion is deskilling (Sambasivan and Veeraraghavan, 2022), i.e., the risk of losing personal abilities in the automated task (in this case, code writing). Similarly, and as already highlighted by previous studies (Sergeyuk et al., 2025), CCTs may increase the risk of generating unsecured code. In response to the challenges that generally affect interaction with AI systems, as in the case of CCTs, a new field of study has emerged in recent years, lying at the intersection of Human-Computer Interaction (HCI) and AI, commonly known as Human-Centered Artificial Intelligence (HCAI) (Shneiderman, 2022; Xu, 2019). HCAI proposes to develop systems that are designed, developed, and evaluated by involving users in the process, aiming to increase performances and satisfaction in specified tasks (Desolda et al., 2024).

While the technical aspects of AI's role in code generation are well-researched (for example, algorithms and AI models used by such tools (Izadi et al., 2024; Husein et al., 2024)), HCAI systems for code completions are still underresearched, exposing developers (and their software) to various risks. Some recent studies confirm these symptoms: only 40% of CCT suggestions are accepted the first time they are proposed to developers, mainly because developers cannot control context or granularity (Liang et al., 2024). Other evidence about the misalignment between developers' mental models and CCTs regards the developers' need for finer-grained suggestions and trustworthy explanations (Sergeyuk et al., 2025; Wang et al., 2023), typically not provided in CCTs, as well as the different preferences about inline and sidebar layouts (Vaithilingam et al., 2023; Mărășoiu et al., 2015). Although some studies explored coders' expectations when dealing with CCTs (Sergeyuk et al., 2025; Wang et al., 2023), the understanding of the cognitive and interaction dynamics that developers face upon adopting CCTs is limited. To the best of our knowledge, no existing studies empirically map the mental models of developers on the interaction with CCTs, models that might determine when code suggestions and explanations are trusted, understood, or disrupted. This aspect limits both theoretical understanding and practical guidance for interaction designers of CCTs. Understanding developers' mental models can thus lead to developing an IDEs that aligns with user requirements, fosters user satisfaction, and improves coding productivity while promoting their adoption (Sergeyuk et al., 2025).

This study addresses the lack of understanding of the user's mental model in the usage of CCTs by reporting

an elicitation study carried out as 8 co-design workshops with a total of 56 participants. Driven by the limitations identified during the analysis of the state of the art, the study aims to answer the following research questions:

RQ1: How do developers mentally model their interaction with AI-driven CCTs?

RQ2: What are developers' expectations regarding the delivery of explanations alongside AI-generated code suggestions?

RQ3: How do developers perceive and articulate their needs for personalization in CCTs?

RQ4: What human-centered design principles emerge for aligning CCTs with developer mental models?

Insights gained from this analysis will help dictate strategies for creating user-friendly interaction approaches in IDEs while increasing cohesiveness among developers and AI technologies in software development.

1.1. Contributions

This work advances the understanding and design of AI-driven Code Completion Tools (CCTs) from a Human-Centered AI perspective by making the following key contributions:

- *Exploratory Survey of CCTs:* An exploratory survey has been conducted to identify and analyze AI-assisted CCTs, their interaction modalities, and explanation features. We contributed in this way by establishing the landscape of available approaches. Moreover, this activity informed the design of our elicitation study, ensuring that workshops prompts addressed both prevalent and underexplored aspects of developers interacting with CCTs.
- *Empirical mapping of developer mental models*: we contribute to the state of the art by proposing the first qualitative elicitation of mental models of CCTs, to explore how developers conceptualize, interact with, and personalize CCTs. Our analysis revealed the multifaceted preferences regarding when code suggestions should appear (manual, proactive, hybrid), where and how they should be displayed (inline, sidebar, popup, chatbot), the granularity and content of suggestions, and the need for contextual, purpose-driven explanations.
- *Actionable design guidelines for human-centered CCTs*: study findings have been distilled into concrete design guidelines, which emphasize that no single interaction model can fit all users or tasks. Indeed, users need highly customizable and adaptive code and explanation suggestions, including per-user control over activation mode, visualization, suggestion scope, explanation detail, and integration with individual and project coding styles.
- *Prototype demonstration of personalizable CCTs*: a prototype of CCT called ATHENA has been developed to demonstrate the feasibility of the proposed guidelines, as well as to provide a platform for future comparative studies on developer-AI co-adaptation.

Together, these contributions provide a significant contribution to the HCI literature on AI development tools, providing evidence and solutions for designing trustworthy, effective, and personally relevant CCTs.

1.2. Organization

The manuscript is structured as follows. Section 2 presents the rationale and background of our study, providing a survey of existing CCTs and their interaction modalities. Section 4 presents the methodology of our study, detailing the co-design workshops. Section 5 reports the findings of our elicitation study. Section 6 discusses the findings, presenting developers' preferences and their mental model of *human-centered* AI-based CCTs. Section 7 illustrates a first working prototype that implements the elicited mental model. Section 8 discusses the elements of the study that could threaten its validity and describes the measures taken to mitigate them. Finally, Section 9 concludes the article, presenting its limitations and potential future work.

2. Rationale and Background

AI-driven CCTs are influencing developers' approach toward programming tasks (Alenezi and Akour, 2025; Martinović and Rozić, 2024). Real-time assistance is provided through these tools with suggestions for code snippets, auto-completion of code syntax, and predicting coding patterns while improving productivity, lowering the cognitive load, and streamlining the coding process. Although the implementation and usability of CCTs very much rely on the developer's expectations, understanding, and workflows, the success of these tools largely relies on how closely they align with what developers are already using, what they expect to use, their skills, and their needs. Achieving this alignment mainly hinges on users' mental models, i.e., internal representations of how the system works.

2.1. Human-centered AI and code completion tools

The rapid growth of AI in recent years has significantly increased its adoption and impact in research and business activities. However, AI's pervasiveness has exacerbated concerns over existing flaws. In particular, biases and lack of explainability endanger the users of the AI models, which often do not consider the human element (Shneiderman, 2020). A report by the National Transportation Safety Board (2017) regarding a deadly crash of an autonomous Tesla car stated the following: «automation "because we can" does not necessarily make the human-automation system work better. [...] This crash is an example of what can happen when automation is introduced "because we can" without adequate consideration of the human element». To mitigate these issues, the novel field of HCAI is gaining traction (Shneiderman, 2022). One of the main goals of HCAI is to produce AI systems that are Reliable, Safe, and Trustworthy (Shneiderman, 2020). For this purpose, AI systems should provide a high level of computer automation while guaranteeing a high level of human control when desired—i.e., AI systems should not automate tasks but amplify, augment, empower, and enhance their users' skills.

In the context of CCTs, systems can be designed either by aiming at *full automation* or at *full augmentation* (or anywhere in between). On one hand, a full-automation approach can provide systems that are extremely efficient, they are associated with several risks. In addition to an increase in commission and omission errors (Cummings, 2004), full-automation CCTs increase the risk of deskilling (Sambasivan and Veeraraghavan, 2022). On the other hand,

previous user studies highlighted that using a full-augmentation approach "by default" may lead to systems that are not on par with users' expectations, thus making them inefficient and ineffective (Esposito et al., 2024). In this context, our study emphasizes the need to design for augmentation, while still aligning CCTs with users' mental models, thus ensuring that developers retain full control over their tools and allowing them to balance the two approaches.

2.2. Usability studies on code completion tools

The usability of CCTs has been a research focus for quite some time, aiming to make these tools more intuitive and helpful for developers. One of the first attempts was carried out by Mărășoiu et al. (2015), which empirically investigated how professional developers interact with CCTs. They analyzed the behaviors, intentions, and obstacles programmers encounter when employing these instruments. The study results revealed that code completion is primarily employed to speed up the coding process and guarantee accuracy. Besides, developers frequently use it as real-time feedback to identify mistakes.

Another aspect investigated by HCI researchers focused on the users' behavior while adopting CCTs. From this study, two main modalities emerged (Prather et al., 2024): in "acceleration" mode, the programmer is aware of their next steps and utilizes the tool to speed up the coding process. In contrast, in the "exploration" mode, the programmer is uncertain about how to proceed and employs the tool to investigate his options. Acceleration mode is usually preferred, as developers often use the tools to complete repetitive code that cannot be copy-pasted (Liang et al., 2024), and for recalling syntax they don't remember (Xu et al., 2022). As a result, the more developers accept CCTs suggestions, the more they perceive themselves as productive (Bird et al., 2022).

Liang et al. (2024) conducted a study to understand the usability of CCTs (such as GitHub Copilot and Tabnine) by utilizing a structured survey distributed to 410 GitHub users who had engaged with CCTs. Questions covered participants' motivations for using or not using AI assistants, the usability issues they encountered, and the strategies they used to optimize the tool's output. The survey revealed that developers are most motivated to use AI programming assistants because they help developers reduce keystrokes, finish programming tasks quickly, and recall syntax. They also found that developers often do not use CCTs because these tools do not generate code that addresses certain functional or non-functional requirements, and because developers have trouble controlling the tool to generate the desired output.

Another important requirement that emerged from user studies is the need for CCTs to learn from feedback and adapt their behavior to developers' styles and project needs. For instance, Zhang et al. (2023) highlight that programmers would like to customize the shortcuts and to set suggestions length and frequency. The literature also highlights the need for explanations (such as reporting the source or linking to documentation) for additional context on the generated code (Liang et al., 2024; Xu et al., 2022).

The proactivity of CCTs has also been investigated, with research encouraging its implementation (Sergeyuk et al., 2024). Vaithilingam et al. (2023) suggested that automatic inline gray-text suggestions, as opposed to traditional lightbulb interfaces, significantly improve discoverability and usability of CCTs. The authors established five design

principles to guide future AI-driven code suggestion interfaces:

- *Glanceability*. Suggestions should be visible at a glance without manual initiation.
- *Juxtaposition*. Clearly displaying original vs. suggested code enhances comprehension.
- *Familiarity*. Using familiar visual elements, like red and green colors for highlighting differences with previous code, simplifies understanding.
- *Visibility for Validation*. Users should be able to view suggestions fully and evaluate them without additional steps.
- *Snoozability*. The ability to temporarily dismiss suggestions reduces interruptions.

Many users have asked for a "snooze" feature that allows them to pause inline suggestions for a specified period or the current session. This points out the need for further research to find the right balance between proactiveness and minimizing interruptions, ensuring that suggestions are presented at the most helpful moments without disrupting the coding process.

2.3. The importance of mental models

A mental model is generally a mental picture users have of how a certain system works (Norman, 1983, 2013). It includes a user's assumptions, beliefs, and expectations while interacting with a system that will lead to their actions, interactions, and problem-solving behavior. Mental models also change over time, might contain errors, and are constrained by the user's prior experiences or technical background. Multiple mental models of a single object may be held by one person to reflect its many functions, and two persons will not always have the same mental model. If mental models are accurate and considered in the design of a system, users can anticipate system responses, feel confident during the interaction, and adapt quickly to new features. On the other hand, gaps between the users' mental model of the system's functionality and its actual behavior can lead users to confusion, frustration, and errors, which will reduce their productivity and satisfaction (Doi, 2021).

CCTs supported by AI employ Machine Learning (ML) algorithms or Large Language Models (LLMs) and rely on probabilistic computations rather than being deterministic. For example, AI-driven suggestions may use aggregated pattern data, which are not directly readable by users. Thus, the suggestions produced might be contextually nonrelevant or impossible to interpret. If a developer's mental model does not match the underlying mechanisms of an AI tool, users might struggle to trust the tool, interpret its suggestions accurately, or recognize when and how to use it effectively.

The need to understand mental models in the domain of AI-based CCTs is multifaceted, and different aspects can benefit from a deep understanding of users' mental models. The first is *interaction* with CCTs since users might predict the tool's behavior and know when a suggestion will be useful and when it could mislead. For instance, a tool

designed considering users' mental models might allow developers to know when the AI might make contextually relevant suggestions and when it can blunder, and so on, to keep the coding flow in hand and avoid wasting time for distraction. Another aspect that can benefit regards *trust*. CCTs can be largely obscure to those operating outside the normal programming logic. It has been proven that transparency and interpretability are essential for establishing user trust, and understanding the user's mental model is essential to making users comfortable when adopting the tools (Desolda et al., 2024). This helps create trust in a tool by aligning design features with users' mental models, making developers more likely to adopt it into their daily workflow. Another aspect that can benefit from the mental model is *cognitive load*: CCTs are meant to reduce repetitive tasks and cognitive load; thus, users can concentrate on more difficult problem-solving jobs. However, if users cannot intuitively understand how or why these suggestions are being made, the cognitive load can actually rise rather than fall. Refining these tools to become less disjointed in relation to user mental models means we can improve the flow of work, minimize interruptions, and ultimately help users be more productive. The last aspect is the possible *support to learning*: especially for novice programmers, developers can recognize common coding patterns and problem-solving techniques, enhancing both tool competency and coding proficiency. The overall improvements in systems that stem from following users' mental models throughout the design are also a potent driver for increasing systems' adoption and acceptability (Sergeyuk et al., 2025). A recent study by Wang et al. (2023) highlights that 54% of the developers recruited for their study are eager for better CCTs.

Exploring the users' mental models is a complex process since these models are typically abstract, partially formed, or even subconscious. Without elicitation studies, bridging this gap is an arduous task, and they are the ones that prescribe structured methodologies to help yield, analyze, and interpret users' cognitive models regarding system functionality. The techniques for eliciting mental models include design workshops (Jacko, 2012), interviews (Rogers et al., 2023), concept mapping (Jacko, 2012), think-aloud protocols (Lewis, 1982), and scenario-based design (Rosson and Carroll, 2012). These techniques allow researchers to design intricate representations of users' mental models of system behavior, what they expect the system to do *a priori*, and how they can expect to interact with, for example, CCTs. Such insights inform tool design and can help bolster our understanding of HCI within AI-augmented coding domains towards supporting frameworks and guidelines for improved user experiences in such environments and beyond.

3. Exploratory survey of AI code completion tools

To inform the design of our elicitation study, we conducted an *exploratory survey* of innovative AI-assisted CCTs. Rather than aiming for comprehensive or systematic coverage, this survey aims to identify tools that are either widely adopted in the industry or that are characterized by distinctive interaction modalities, relevant to our research questions. We identified candidate CCTs through a review of recent academic publications, developer forums, official product websites, and prominent marketplace listings (e.g., Visual Studio Marketplace, GitHub repositories) as of April 2024. CCTs were included if they provided a graphical user interface and exposed features for code sugges-

tion and/or explanation. We focused on selecting those CCTs that represented the breadth of current interface and explanation approaches encountered by developers in practice.

The principal goal was not to exhaustively catalogue the CCT landscape, but to develop a functional mapping of interaction paradigms and explanation methods. This mapping directly informed the design of our empirical study by grounding scenario construction and workshop prompts in real, contemporary CCT examples. While acknowledging limitations inherent in a non-systematic approach, this selection ensures our study addresses key facets of developer–CCT interaction as manifested in actual products.

3.1. Code completion tools analysis

The proliferation of powerful AI tools, such as LLMs, is stimulating the creation of many CCTs. The analysis of the literature and the market allowed us to identify a total of 17 tools. Each tool is characterized by many aspects, ranging from technical features (e.g., the underlying AI model) to more social aspects, such as interaction with them. Given the purpose of this study, we analyzed the 17 CCTs with respect to the interaction techniques offered. However, 4 do not provide a graphical user interface, so they are not considered in the following.

The analysis of the remaining 13 CCTs providing a User Interface (UI) was guided by two main aspects: the *code suggestion* itself and the *explanation*, if any, of why that code was suggested. After familiarizing ourselves with the CCTs, we analyzed users' interaction with them across different dimensions. To simplify the analysis and the reporting of the results, we used the 5W model by Lasswell (1974), which is adopted in various fields, such as journalism and customer analysis, and more generally in problem-solving, to analyze the complete story of a fact. A similar approach is employed in the field of HCI, where the 5W model is utilized for mental model elicitation, and comprehensive user analysis (Jacko, 2012; Desolda et al., 2017; Duric et al., 2002). Each dimension of the interaction with the tool was addressed in terms of the question of the 5W model. In this phase, the 5W model guides the analysis of the state of the art of CCTs. The questions used in this phase are similar to those presented in Section 4.3, which are used to elicit users' mental model: this choice allows the comparison of existing CCTs with users' expectations. This choice allows to leverage existing tools (identified during this analysis) as design inspirations during prototype development (see Section 7), since it allows the potential identification of tools that satisfy users' expectations (elicited in the co-design workshops, see Section 4). The analysis of the tools with respect to the code suggestion was done by answering these questions:

1. *Where* to show the code completion in the UI?
2. *When* should the suggestion be activated during the interaction?
3. *How* should the suggested code be shown?
4. *What* should be suggested?
5. *How* should the suggestion be customized?

Table 1: Summary of the analysis of interaction with code and explanations generated by 13 Code Completion Tools.

	Code completion					Explanation		
	What	**When**	**How to show**	**How to customize**	**Where**	**What**	**When**	**Where**
Intellicode (Svyatkovskiy et al., 2020)	lines	proactive	grey text	curent file	inline	-	-	-
Codex (Chen et al., 2021)	lines	manual	text	chat history	right side	answer	prompt	chatbot
Copilot (Friedman, 2021)	lines	proactive	grey text	current file	inline	answer	prompt	chatbot
TravTrans (Kim et al., 2021)	token	proactive	dropdown	current file	inline	-	-	-
StarCoder (Li et al., 2023)	lines	proactive	grey text	current file	inline	-	-	-
Codegeex (Zheng et al., 2023)	lines	proactive	grey text	current file	inline	answer, description	prompt, click	chatbot
CodeWhisperer (Amazon, 2024)	lines	hybrid	grey text	current file	inline	source	click	sidebar
Codeium (Codeium, 2024)	lines	proactive	grey text	current file	inline	improvements, reasons	click	chatbot
Replit (Replit Inc., 2024)	lines	proactive	grey text	current file	inline	description	click	pointer
Cody (Sourcegraph, 2025)	lines	proactive	grey text	repository	inline	answer	click, prompt	chatbot
Tabnine (Tabnine, 2025)	lines	proactive	grey text	current file	inline	answer	prompt	chatbot
Xcode (Apple Inc., 2024)	lines	proactive	grey text	current file	inline	-	-	-
Blackbox (Blackbox AI, 2024)	lines	proactive	grey text	current file	inline	improvement	click	sidebar

The analysis regarding the explanations was guided by the following questions:

6. *Where* to show the code completion explanation in the UI?
7. *When* to show the explanation during the interaction?
8. *What* should be explained?

A summary of the complete set of such models is depicted in Table 1.

3.2. Code suggestions

An important characteristic affecting the interaction with CCTs is the timing to provide suggestions (*when*). The analysis of the existing tools revealed two main approaches: *proactive* and *manual*. Proactive CCTs provide suggestions at a certain time without requiring a user action to trigger the completion. This type of design can be either implemented in an intrusive manner (by suggesting a completion at any time) or discreetly (by generating completions only when the context is sufficient). The manual approach, in contrast, requires the user to perform a specific action to visualize the completion. However, manual activation of CCTs is not very popular, as it is adopted in only two tools. A hybrid timing approach is also implemented, even if only in the case of CodeWhisperer, as completions are proactive but can be forced through a shortcut (Amazon, 2024).

Another aspect is the content of CCTs suggestions (*what* is provided as a suggestion?). Two main modalities emerged from our analysis: *single token suggestions* (less popular, as it was proposed only with the TravTrans prototype (Kim et al., 2021)) and *multiple line completions*, which are widely adopted. While the first solution is quite simple, the second one results in a feature that is hard to control and assess since the length of the suggestions may

vary depending on the context and LLM characteristics. As a matter of fact, multi-line suggestion length can range from short code blocks (e.g., for loops, switches, if-else) to entire functions or classes.

The suggestion's location (*where*) in the UI is also worth exploring. A predominant approach is to put completions directly inside the code (*inline suggestions*). The other solution instead consists of providing the suggestion in a *separate space* with respect to the coding area. Codex (Chen et al., 2021) is the only tool that adopts the latter modality.

It is useful to discuss *how to show* code completions that appear before they are accepted and integrated into the live code. A consolidated approach is to display the completed code in gray, allowing users to easily distinguish it from the live code, which is typically shown in white (or other colors, depending on the users' preferred color scheme), before accepting the proposals of the CCT. Another method that emerged from the analysis is displaying multiple possible completions in a dropdown menu, which may be particularly convenient for single-token suggestions or very short completions, as shown by TravTrans.

Finally, it is worth investigating the customizability of CCTs, as the relevance and helpfulness of suggestions strongly depend on the type of user receiving them (*how to customize*). Context-aware suggestions may consider the user expertise, the specific formatting/adherence to coding standards, and the functional and non-functional requirements of specific software development scenarios to provide the most accurate code snippets possible. The strategies adopted for this aim result in three categories: personalization based on the context of the current file, personalization based on a code repository, and personalization based on chat history. The first category is the most popular, while Cody (Sourcegraph, 2025) is the only tool that managed to include a code repository for more relevant completions. Nevertheless, this last approach seems to fail for big repositories. Codex provides more aware suggestions by processing the conversation between the programmer and the chatbot as context.

3.3. Explanations of the suggested code

Some of the analyzed CCTs do not offer any explanatory features. By contrast, the remaining tools use various methods to provide developers with explanations as they write new code.

The location in the UI (*where*) of code explanations is an important aspect of CCTs, as it influences other interaction characteristics. Three main approaches are observed: explanations are either delivered inside a chatbot, or shown in a sidebar, or finally displayed inside the coding space. Tabnine (Tabnine, 2025), Cody (Sourcegraph, 2025), CodeGeeX (Zheng et al., 2023), Copilot (Friedman, 2021), Codeium (Codeium, 2024), and Codex (Chen et al., 2021) implement the first approach, providing chat-based explanations. In contrast, CodeWhisperer (Amazon, 2024) and Blackbox (Blackbox AI, 2024) report code explanations in a sidebar. In this case, the user is not allowed of submitting prompts for additional details, but such an approach delivers code explanations quicker, as the user does not have to think about a prompt in order to receive explanations. Finally, Replit (Replit Inc., 2024) follows a unique approach, showing explanations inside a tooltip at the mouse pointer's location (which means the explanations appear within the coding space).

As mentioned earlier, the position in which explanations are delivered directly impacts their content (*what*). Chat-based explanations typically offer direct responses to specific user questions. Static explanations, in contrast, can vary widely. For instance, CodeGeeX and Replit provide descriptive text for code, with Replit defaulting to a short, numbered list format, as too much text would be too intrusive to fit inside the coding space. CodeWhisperer lists sources used for generating completions, and Blackbox and Codeium suggest code improvements, with the latter adding implementation rationales to its suggestions.

The timing approaches (*when*) are exclusively manual for explanations. Two primary modalities emerge: explanations can be triggered by a mouse click or provided after submitting a prompt (in the case of chat-based explanations). CodeWhisperer, Codeium, Replit, and Blackbox follow the former approach, although multiple clicks are often required to access the explanations. The latter approach is used by Codex, Tabnine, and Copilot. CodeGeeX and Cody adopt a hybrid approach, offering explanations through both methods.

4. Methodology

This section details the methodology that was employed in this study. Leveraging the 5W model by Lasswell (1974), we conducted co-design workshops that aimed at eliciting users' mental models and expectations for CCTs.

4.1. Research design

This study investigates the mental model of developers interacting with CCTs. Based on the results and limitations that emerged during the literature analysis, we address the following research questions:

RQ1: How do developers mentally model their interaction with AI-driven CCTs?

RQ2: What are developers' expectations regarding the delivery of explanations alongside AI-generated code suggestions?

RQ3: How do developers perceive and articulate their needs for personalization in CCTs?

RQ4: What human-centered design principles emerge for aligning CCTs with developer mental models?

To answers these questions, we followed an elicitation study methodology conducted through design workshops. We have chosen this technique because, particularly when performed in workshop formats, it facilitates in-depth insights into users' thought processes, preferences, expectations, needs, and perceptions about complex systems (similar to what has been done by Desolda et al. (2017); Marquardt et al. (2012); Voida et al. (2005)). Moreover, workshops stimulate participants to collaboratively propose, discuss, and model their ideas on code completion tools, offering rich qualitative data suited to the study's objectives. To structure the workshop discussions, we adopted a scenario-based design approach. This solution helps participants provide their answers while referring to a realistic situation (Rosson and Carroll, 2012).

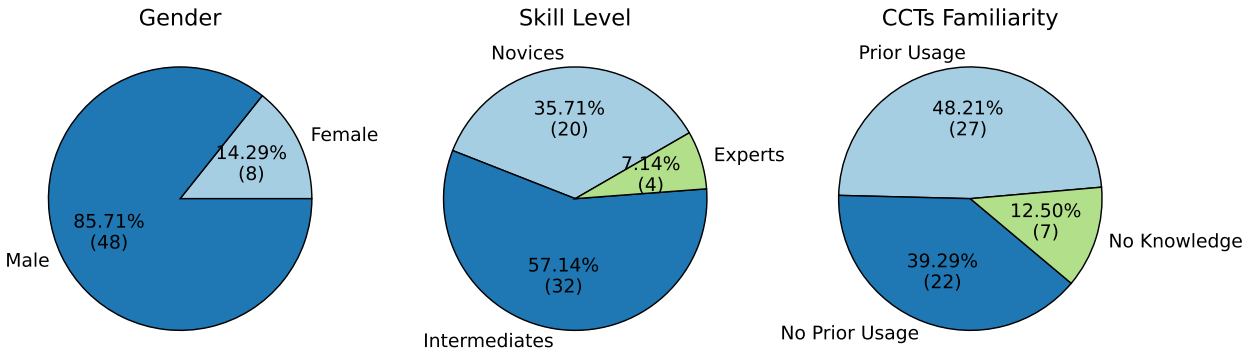

Figure 1: Participants' demographic information

4.2. Participants

The elicitation study involved a total of 56 participants (8 female, 48 male), aged between 20 and 30 years (M = 22.88, SD = 2.63). They were randomly divided into 8 groups of 7 participants; this group size was chosen to enable balanced discussions where each participant could contribute their perspectives while fostering dynamic group interactions. Figure 1 presents participants' demographic information. The participants were students from both the University of Bari and the University of Salerno; 4 already had bachelor's degrees in computer science, while the remaining were students in the third year of their bachelor's degrees in computer science. Participants included novice (20), intermediate (32), and expert programmers (4). Their programming experience ranged from 1 year to 8 years (M = 3.36, SD = 2.07), with familiarity in languages such as C/C++ (most common ones, as 100% of participants were familiar with them), followed by Javascript (known by 66% of participants), Python, Java, and C#. A total of 27 participants used CCTs; 22 participants had never used CCTs, but they knew what CCTs do, while 7 participants had never used and heard about CCTs.

All relevant ethical guidelines and regulations were followed, and the study received ethical approval from the Independent Ethical Review Board of the Computer Science Department of the University of Salerno. The participants digitally provided their informed consent and could leave the study at any time.

4.3. Procedure

Two weeks before the start of the workshops, 100 emails were sent to the students inviting them to participate in the study. A total of 58 students agreed to attend the study, but then 2 of them missed the workshops. Those who signed up voluntarily filled in a questionnaire asking for personal data (first name, last name, age, gender), and data on technical skills (self-rating programming experience as novice, intermediate, expert), number of years of experience as a programming professional, use of CCTs ("yes", "No, but I know what they are and how they can work", "No, and I don't know what they are"), and used programming languages. We also surveyed their availability for the week of the study to schedule their participation according to their preferences.

Two HCI researchers performed the study on four consecutive days in a quiet university laboratory; one was the conductor, and the other acted as the observer. The entire study consisted of 8 sessions, one for each group. Each workshop was planned to last around 1 hour and comprised four stages: introduction, scenario exploration, mental model elicitation, and debriefing.

For each session, the group sat around a table and was provided with paper sheets and markers to sketch their proposals. One of the two HCI researchers began by welcoming participants and thanking them for attending the study; then, the conductor gave a 5-minute presentation to provide an overview of the study's objectives, ensuring that participants understood the focus on user interaction with CCTs. Participants were also informed of the confidentiality and voluntary nature of the study.

To allow participants to recall the main concepts of CCTs (if they already used them) or to familiarize themselves with them, we presented a scenario involving a typical developer named Andrea, employed in a company producing web applications. In this scenario, Andrea has access to new functions of the IDE for which an AI-based module can suggest code to help him in his work. No concrete ideas or possible solutions were reported in the scenario to avoid bias in the participants' proposals.

After the introduction and the scenario presentation, the workshop's main phase began. The conductor led the discussion, focusing on those critical aspects of the interaction with CCTs that emerged from the literature review. In particular, for code completion, we posed the following questions:

1. Where to show the code completion in the UI?
2. When should the suggestion be activated during the interaction?
3. How should the suggested code be shown?
4. What should be suggested?
5. How should the suggestion be customized?

For the explanation part, the discussion was guided by the following questions:

6. Where to show the code completion explanation in the UI?
7. When to show the explanation during the interaction?
8. What should be explained?

For each question, the participants were asked to reflect individually for about 30 seconds and present the solution they had thought of to the others; afterward, the group was driven by the conductor in a discussion to come up with one or more ideas to answer the specific question.

Once all the questions had been answered, the session ended with a debriefing to summarise the key points raised by the participants and to revise, if necessary, some of their answers in the light of what had emerged throughout the discussion.

4.4. Data collection

The data collected during the workshops were 1) the notes taken by the researchers, 2) the audio recordings, and 3) the sketches drawn by participants. The two researchers transcribed the notes and audio recordings, and independently rechecked 80% of the material. The initial reliability value was 70%; then, the researchers discussed the differences and reached full reliability.

The researchers analyzed the transcripts systematically using an inductive thematic analysis (Braun and Clarke, 2006). According to the thematic analysis protocol, the evaluators performed the following 6 steps: familiarizing with the data, generating codes, mapping codes that share similar meanings, reviewing potential themes, defining and naming themes, and producing a report. The eight guiding questions structured the analysis, with the final themes answering each question.

5. Findings

This section presents the lessons learned about the user mental model on code completion for both suggestion and explanation, drawn from the elicitation study. The findings are presented with respect to the questions that structured the discussions during the workshops and provide a concrete description of users' thoughts about desirable interaction with code completion and explanation features. Each finding derives from a single theme that emerged from the thematic analysis, which is, in turn, supported by one or more low-level codes. All the themes, codes, and frequencies are reported in Appendix A.

The names of each theme are presented in italics in the following subsections. Participants' answers are given in double quotes to illustrate how one of the answers led to the identification of a theme. We also specified the number of groups that proposed the solution linked to each theme. It is worth noting that we chose to report the number of groups rather than the number of participants because, within a group, several solutions for each question were initially proposed individually; however, subsequent discussion among group members led to the identification of one or more solutions to answer the question.

5.1. UI for code completion

The results of this subsection address RQ1, since it focuses on how developers mentally model their interaction with AI-driven CCTs. The findings describe developers' preferences on when and how code suggestions should be triggered, displayed, and customized, in order to adhere to users' internal representations and expectations toward CCT behavior. The themes, represented in Fig. 2, also partially cover RQ3, specifically on the developers' needs for customizing activation and display strategies.

Figure 2: Themes emerged by analyzing the developers' answers to the questions on user interfaces of CCTs.

When to show suggestions. There were differing proposals with regard to when the CCTs should provide code suggestions for participants. Three main strategies emerged:

- **Proactive** (4 groups): the CCT automatically suggests the code without users' requests when the user is not actively typing, e.g., coding 'breaks' in the coding flow ("*When someone is blocked, there a suggestion is given to help me*" – G1)
- **Manual** (5 groups): activation by keyboard shortcut or mouse interaction ("*It must be activated manually via a shortcut*" – G2). This solution leaves complete control to the user over the use of AI, which could then be less intrusive.
- **Hybrid** (6 groups): a basic behavior could involve a proactive AI strategy that intervenes only when there is a well-defined context (e.g., a long code already written) and when the suggestion is almost taken for granted and

difficult to discard by the user (e.g., completion of a control structure). On the other hand, manual intervention would compensate the user for the lack of automatic AI intervention, provided that the user is sure that they require AI support ("*Proactive when it's really obvious and useful, or when the user explicitly requests it*" – G8). Furthermore, several participants specify that, in general, suggestions should be generated only if there is enough *coding context available* (4 groups) ("*It's better not to try immediately, but take from context*" – G2). This was also consistent with the perceived necessity of very precise, context-sensitive recommendations (Asaduzzaman et al., 2014).

How to show suggestions. The second aspect concerns how to show the suggestions. Four suggestions emerged:

- **Inline with distinct styling** (5 groups): code suggestions should be displayed within the main coding window, but with differences in formatting, such as unique font styles or colors ("*... chosen the 'phantom' code suggested, it is then shown in our code, to give a 'preview'*" – G6). This method allows users to view suggestions in context while maintaining clarity, particularly when the suggested code is short ("*For little code, it is convenient showing it intext*" – G2). In addition, a button could be useful to switch this display on or off, i.e., to show AI and user-written code in the same or different fonts ("*... turn on/off the 'show AI/human written code' mode...*" – G8).

- **In a separate area** (7 groups): visualization of suggestions in a separate part from the text editor, such as a sidebar. This modality allows users to review suggestions (especially when they are lengthy or more than one) without cluttering up the main coding space while keeping the focus on the existing code ("*A separate window with a list of suggestions*" – G4; "*For on-demand things or lengthier suggestions, use a separate window*" – G3).

- **Clicking on an icon** (2 groups): manually activate suggestions by clicking on a button and displaying it in a popup near the line where the suggestion is applied. By clicking on the icon, users can see and confirm the insertion of the suggestion, minimizing interruptions and increasing the level of control ("*Maybe a 'lightbulb' icon that allows opening a popup*" – G3).

- **Using chatbots** (2 groups): allow users to request suggestions in a conversational manner. The suggestion would then be entered directly into the IDE when the user accepts the chatbot's recommendation ("*It should be conversational to improve suggestions*" – G3).

Where to show suggestions. The third aspect concerns where to display the suggestion in the user interface of the IDE. Discussions with participants led to four strategies:

- **In a sidebar** (6 groups): the first preference expressed by participants is to show the suggestion in a sidebar ("*It's better on the side, separately*" – G1; "*In a movable tab of the IDE*" – G7).

- **in a pop-up** (3 groups): the second strategy suggested by participants is to show the suggestion in a pop-up near the main code ("*In a togglable pop-up near the main code*" – G3).

- **Inline**(5 groups): another strategy that participants came up with is to show the suggestion inline with the code that the developer is writing; in this case, the user could also be given a clear and immediate option to accept or reject the suggestion, e.g., an accept or reject icon ("*[the suggestion] is shown within the editor, with the possibility to confirm the insertion"* – G8).

- **Adaptive visualization** (2 groups): the last mode combines the previous ones in which shorter suggestions appear inline and longer ones are presented in a separate window ("*In loco or in a different area. The choice depends on the code quantity"* – G2). This approach allows flexibility depending on the length and complexity of the suggestion.

What to suggest. The fourth aspect concerns what to show in the suggestion. Participants expressed different expectations of what constitutes valuable content in code suggestions:

- **Suggestions of various granularity** (7 groups): in general, the strategy of controlling granularity emerged. Notably, participants expressed very contrasting needs for different levels of granularity for code suggestions: while some groups agreed on preferring suggestions up to a maximum granularity level of *function* ("*Limit to completing the function: if it completes everything, randomness increases and precision decreases*" – G1), others agreed on the usefulness of generating up to entire code files or classes ("*Potentially a class*" – G7). Some participants proposed inserting a slider at suggestion time to control the level of detail of the suggestions, allowing developers to switch from minimal to very long suggestions according to their needs ("*Let the user choose what to suggest, for example with a slider*" – G2).

- **Minimal suggestions with details asked on-demand** (4 groups): there was no single agreement on the minimum number of tokens that should be suggested; however, there was a certain agreement on preferring, initially, minimal suggestions with details asked on-demand to foster the user's reasoning ("*Only suggest the function name to insert, so that [the developer] can solve the problem on their own [...] pieces of code with comments that allow us to receive cues/hints; do not suggest too much code.*" – G6).

- **Different alternatives** (3 groups): the aspect of having different alternatives offered by the code completion tool was also brought up as a preferable feature to improve precision in suggestions, particularly in the case of coding challenges that the developer has never faced ("*Better to have many suggestions for new things, but for more 'precise' things it's better to have single suggestions*" – G1).

- **Code accompanied by documentation** (2 groups): a cross-cutting aspect of the above concerns the presence of documentation together with the code to improve comprehension and facilitate faster decision-making about the acceptance or rejection of the suggestion ("*Include references to documentation for functions*" – G5).

- **Code adapted to the context** (3 groups): code should be aware of the context in which it is suggested , including variable names or coding styles relevant to the current project. According to the participants' proposals, the

contexts could be represented by the developer's profiles (e.g., previous projects stored in repositories) or the projects already developed by the company ("*[define] the standard of the suggested code, the style (in line with my style, or with the style of my company, of other repositories)...*" – G4).

How to customize suggestions. The last aspect investigated during the study concerned how to customize the suggestions. Several suggestions emerged:

- **Personalize activation timing of suggestions and explanations** (6 groups): the first need proposed by users concerns personalizing *when* to suggest explantions to allow each developer to tailor the responsiveness of the tool to their own workflow ("*The activation timing, to disable/enable automatic hints*" – G5). The need of customizing when to show explanations also emerged to adapt to different expertise levels of programmers ("*Automatic generation of explanations is fine for novices, but could be obnoxious or even 'outrageous' for experts*" – G2).

- **Customize appearance** (6 groups): another aspect that emerged in this dimension concerns customization of appearance: for example, many participants emphasized the need to customize the fonts, colors, and text size of the suggestions to ensure that they are visually distinct from the user's code according to their preferences ("*Customize character size, color, font of suggestions..*" – G4).

- **Customize granularity of suggestions and explanations** (6 groups): in the What dimension, we reported the strategy of controlling granularity, which generally prescribes proposing the suggestion with different levels of granularity. Here emerged that users should be able to customize this option, e.g., the user could choose a 3, 4, 5 level scale of granularity, and for each of them, users can choose what should be suggested, e.g., the statement the user is writing, the control structure, the function, the class, or others ("*Set the level and scope of suggestions (the context of action – functions, loops, operators)*" – G7). The need for customization also emerged for the granularity of explanations, to adapt to the user's knowledge by generating more or fewer comments to explain the suggested code ("*... the presence of automatic comments and their quantity/frequency (every line, every function, etc.)*" – G5; "*Customize what is generated as explanations... Customization can be simple, such as a 'what level of programming are you?' initial question*" – G6).

- **Tailor coding style** (4 groups): participants stated that developers should adapt the tool to a certain coding style by fine-tuning it by feeding in, e.g., some of their own code repositories or their companies' past projects as examples ("*Which context to use in the code generation as a 'background training' ... set what repositories to consider*" – G2). Moreover, the developers should also be able to define quality standards to which the suggested code should adhere ("*Control the quality of generated code, adherence to standards (even security ones)*" – G4).

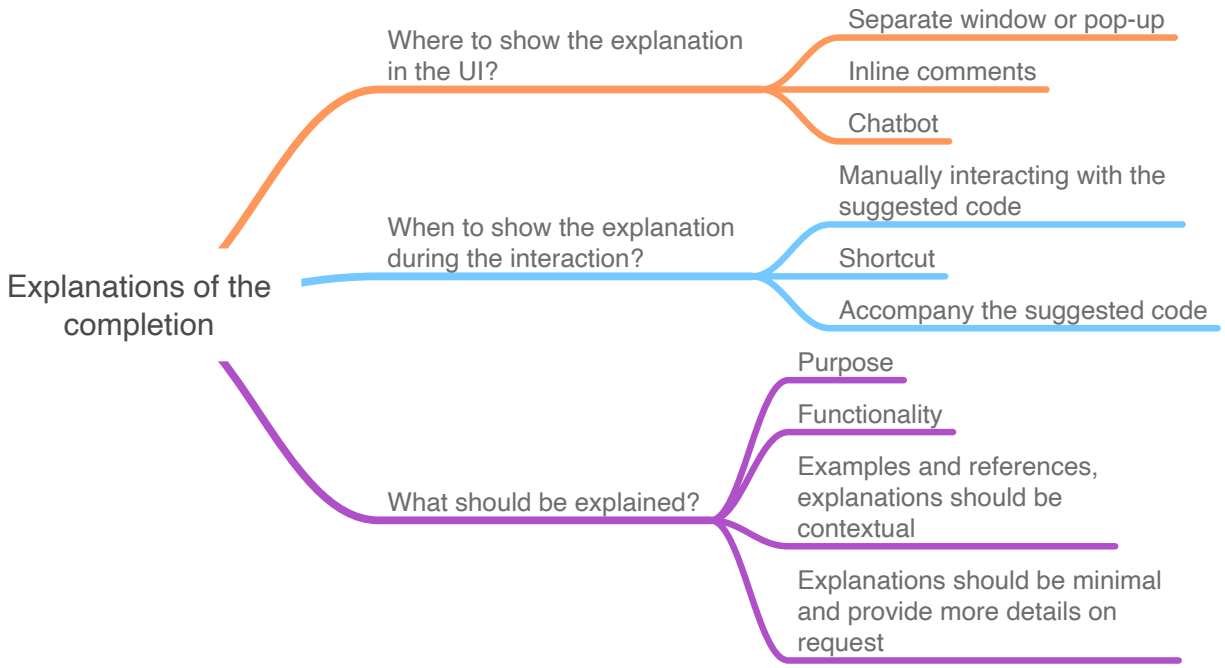

Figure 3: Themes emerged by analyzing the developers' answers to the questions on explanations produced by CCTs

5.2. *Explanations of the completion*

The results of this subsection answer RQ2, which investigates developers' expectations on how explanations should be delivered by CCTs. The study findings clarify user preferences about the timing, format, and content of explanations. In addition, the need for a customizable level of detail and contextual delivery answers RQ3, highlighting the importance of tailoring explanations to user expertise and task demands. Figure 3 represents the identified themes.

Where to show the explanation in the UI. The first aspect we analyzed regarding suggested code explanations concerns **where** the explanations should be displayed in the user interface of the CCTs. Three suggestions emerged:

- **Separate window or pop-up** (7 groups): several participants proposed that the explanations are better suited to a separate interface element, such as a sidebar or pop-up, allowing them to access the explanations without cluttering up the coding work area ("*The explanation and the function must be shown separately, otherwise there's too much text*" – G1).

- **Inline comments** (6 groups): another strategy concerns visualization employing inline comments within the code, thus providing an immediate context for the suggested code without having to navigate elsewhere ("*As an expandible text under the code"* – G7); however, this solution was indicated as viable only if the explanation length was limited, in order to avoid confusion in the code ("*No comments, as they tend to be too long... but a 'title' of the suggestion can be kept as a brief explanation in a comment*" – G6).

- **Chatbots** (3 groups): some participants suggested accessing the explanations via a chatbot, allowing users to

query the system in a conversational manner to obtain further clarification of the suggestions (*"An alternative could be the chatbot to which I ask why [the code was suggested] after selecting the code"* – G8).

When to show the explanation during the interaction. Three complementary strategies emerged concerning the timing of explanations, i.e., when explanations should be shown during the interaction with the CCTs:

- **Manually interacting with the suggested code** (6 groups): the first solution is to activate the explanations by manually interacting with the suggested code. For example, users can select the piece of suggested code they want to explain, right-click, then choose an ad-hoc function from a contextual menu, and the explanation is then displayed in a separate window, in the chatbot, or as a comment ("*For example with the right click on the code and then pressing 'explain'*" – G1); hovering over suggested code to show explanations contextually was also proposed by some groups ("*... and then with an explicit request ... hovering on the code*" – G6).

- **Shortcut** (3 groups): an alternative solution concerns activation via shortcuts. Indeed, some participants suggested activating the explanations through a dedicated keyboard shortcut, granting users control without interfering with their workflow ("*With a key I invoke the explanation*" – G8).

- **Accompany the suggested code** (3 groups): an automatic approach along with the suggestion has also been proposed, i.e., also accompanying the suggested code with an automatic short explanation, expandable on-demand, which is especially useful when the suggestion is complex, reducing cognitive effort by presenting both simultaneously. A longer suggestion should be provided on-demand, for example, by interacting with the short suggestion and invoking more details ("*... generated together with the code, as long as it is brief*" – G2; "*[explanation] details are provided on-demand*" – G3).

What should be explained. The last aspect concerns what to explain. The desired content of the explanations was varied, with participants identifying three key elements:

- **Purpose** (5 groups): explain why a certain code suggestion was given. Indeed, many participants emphasized the need for explanations that clarify the purpose of the suggested code, specifying what it achieves in the broader context of the program ("*[explain] why did it give me this suggestion?*" – G6; "*[explain] pros and cons of a suggested technique*" – G8).

- **Functionality** (4 groups): describe how the suggested code fulfills its purpose by reporting technical details such as the data structures, code constructs, data types and syntax involved ("*What is the code composed of (in terms of data structures and chosen constructs)?*" – G6; "*explain the code, data types, ... syntax*" – G3); some groups also suggested displaying flow diagrams or pseudo-code to help developers understand complex suggestions and visualize the inner workings of the generated code ("*With pseudocode, maybe also flowcharts*" – G1).

- **Examples and references** (5 groups): another popular aspect concerns the use of examples and references to sources : participants recommended that explanations include examples of the use of similar codes or references to reliable sources that might enable users to assess the credibility of the suggestion ("*It may be sufficient to provide simple 'pointers' to something that helps me understand ... such as links to source codes, or similar code examples as 'insights'*" – G2).

- **Explanations should be contextual** (4 groups): regarding the scope of explanations, they should be contextual and not regard the entirety of the suggested code, but only the pieces of code that the user indicates ("*We're not interested in being explained everything necessarily, but only of a piece [of code]*" – G1).

- **Explanations should be minimal and provide more details on request** (4 groups): regarding the length of explanations, there was a popular agreement that explanations should be minimal and provide more details on request. Participants suggested that this can be achieved by, e.g., clicking a button to expand the explanation preview or by interacting with a chatbot to obtain an explanation incrementally ("*If something else specific is needed, the user can ask explicitly*" – G3).

6. Discussion

Our findings highlight important points regarding developer preferences and requirements while working with AI-driven CCTs. In particular, this work underlines the importance of designing CCTs for developers' mental models to promote productivity, efficiency, and an intuitive coding experience. Figure 4 summarizes users' preferences and requirements, attempting to elicit developers' mental model of CCTs.

This section interprets and presents the findings of our study in light of the research questions. Specifically, Sections 5.1 and 5.2 addressed RQ1, RQ2, and RQ3 by identifying how developers mentally model their interactions with AI-driven CCTs, what they expect from explanations, and how they express their need for personalization. More broadly, this section addresses RQ4 by distilling the study results into actionable human-centered design suggestions, which are intended to guide the development of CCTs that align with developers' mental models and support diverse coding workflows.

6.1. Mental model of activation and control (RQ1)

Participants expressed different preferences regarding when code suggestions by CCT should be triggered, namely proactive, manual, and hybrid activation. This aspect is supported by the results detailed in Section 5.1 ('When to show suggestions?') and Table A.3, where proactive activation was favored by 4 groups, manual by 5 groups, and hybrid by 6 groups.

This diversity in the mental model points to a key insight: in different contexts and coding tasks, developers might prefer different degrees of control. Some users mentioned that proactive suggestions, automatically activated during the coding flow breaks, might increase coding speed without explicit request. However, most users expressed the

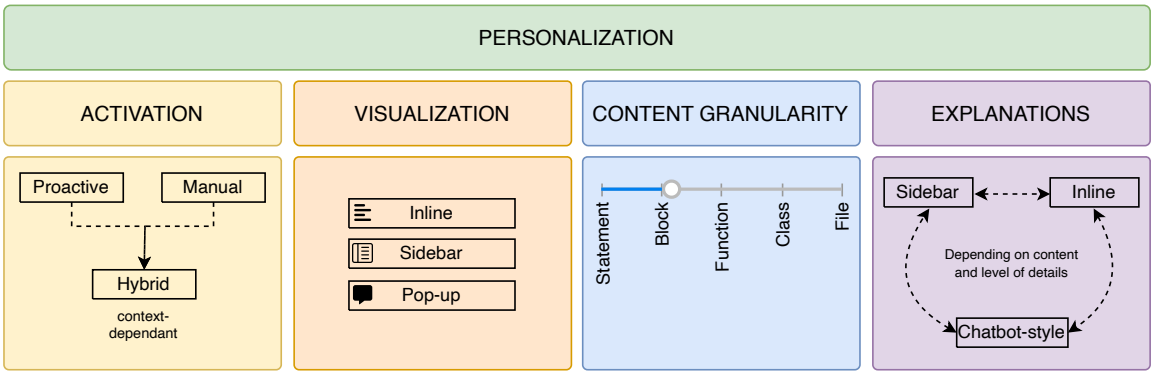

Figure 4: Graphical representation of users' preferences and mental model. The four aspects of CCTs, namely activation, visualization, content granularity, and explanations, are shown. All these dimensions are relevant for users' personalization.

need for activation on-demand to control exactly when suggestions appeared. This is in line with similar studies regarding activation, as frequently CCTs are not adopted as they might disrupt developers' workflow (Sergeyuk et al., 2025). Rather, users favored a hybrid approach in which proactive behavior occurs only when a clear contextual need is present (such as when completing a complicated bit of code). However, proactive behavior should only occur once there is enough typing context (e.g., suggest code only if the file contains X tokens or more) to avoid generating imprecise or random suggestions. Therefore, a dynamic and adaptive activation model could serve different user needs by altering its behavior in response to task complexity and user-characteristic usage patterns. Such a model would leave developers in control of their workflow while helping proactively when needed.

6.2. Visualization preferences (RQ1)

The visualization mechanism of suggestions impacts the usability and acceptance of CCTs. The participants proposed different visualization strategies, each with pros and cons. As shown in Section 5.1 ('How to show suggestions?') and summarized in Fig. 2, the majority of groups (5/8) preferred inline suggestions, as well as suggestions displayed in sidebars (7/8). Having inline suggestions can be useful to avoid losing the coding focus and allow the user to accept or reject the suggestion immediately. However, the inline solution is well-suited for short and concise suggestions that do not interfere too much with the code manually written by the users; on the contrary, in the case of long and complex suggestions, it may be better to have separate sidebars to limit visual clutter. A further level of control would be having users manually invoke popups containing the suggestion by clicking an icon shown close to the handwritten code (proposed by 2 groups). Moreover, a chatbot in a separate window could be used as an alternative way to suggest code on demand (proposed by 2 groups), with all the attached benefits of using conversational agents for code generation (Ross et al., 2023).

All these results may reveal an *adaptable visualization*, where users can choose or alternate between inline, sidebar, and popups, and an *adaptive visualization*, where the system visualizes the suggestion according to factors such

as the suggestion length. Such modalities can improve usability by adapting to different workflows and developer preferences and might reduce the errors due to wrong intention communication as multiple choices may be provided when needed (Sergeyuk et al., 2025). This flexibility can also lower the cognitive overhead caused, for instance, by long pieces of generated code shown next to user-written code; in this way, CCT allows developers to adjust the display of suggestions according to the complexity of the coding task or their personal coding style.

6.3. Granularity and content of suggestions (RQ1, RQ3)

The study findings revealed that CCTs should be capable of generating suggestions at varying levels of granularity and that this granularity must be in the control of the users. This finding traces to Section 5.1 and Table A.5, where it is reported that 7 out of 8 groups supported user control over suggestion granularity, including requests for minimal suggestions with additional detail on-demand found in 4 groups.

Participants did not clearly agree on the minimum or maximum number of tokens that should be suggested at any given time. Consequently, the CCTs should be designed to produce suggestions at different scopes, from predicting the next token in the code to completing control structures and generating entire functions or classes.

Although longer suggestions are useful, they can overwhelm the developer if displayed without considering the task context. For this reason, participants prefer retaining control over the level of granularity of suggestions. The necessity for varying degrees of granularity is also shared by other participants in different studies (Wang et al., 2023). However, in general, users expressed a preference towards minimal, "lower-scope", suggestions at the first moment, with the possibility to obtain a longer, "higher-scope", suggestion on-demand; this approach of "letting the user reason" could help to prevent overreliance and deskilling. In practice, these directives might translate concretely into a minimalist suggestions behavior by default, with the possibility of using a slider or a control mechanism to adjust granularity in real time and adapt the suggestions to the contextual needs of the developer.

The control of granularity should also consider the number of offered alternatives; in certain use contexts, such as when developers face coding problems that they are not confident in, it may be beneficial for the user to have multiple options. Furthermore, code suggestions in such contexts may include documentation for constructs or functions the developer is not used to handling. Conversely, a suggestion with limited or no accompanying documentation should be presented for more straightforward and routinary tasks. These aspects could enhance user trust and overall acceptance of the suggestions.

6.4. Preferences for explanation delivery (RQ2)

Participants generally considered explanations useful, but the answers to the question of *what* should be contained in an explanation revealed a need for special care in their design. The concept of what defines a good explanation is not trivial and is an aspect that is widely studied in the literature (Miller, 2019; Holzinger et al., 2020). Moreover, even well-designed explanations should be evaluated with users to measure their quality (Holzinger et al., 2020). This elicitation study can be a precious source of information in understanding the desiderata of explanations in CCTs

from the developers' point of view. A key finding is that explanations should be classified into two categories: *why-explanations* and *what-explanations*. Why-explanations address the rationale or purpose behind the generated code, whereas what-explanations concentrate on the functionality and technical intricacies of the code suggestion. The content should, however, be minimal, with the possibility to obtain more details when requested. This might translate into the CCT by initially showing a brief why-explanation, providing a bird-eye view of what the generated code is for. A more thorough, on-demand explanation would instead detail the code nuances and technical aspects to help the developer get a deeper understanding of the suggestion, possibly even with flowcharts. Detailed explanations should also include references to trusted sources, such as documentation snippets, book pieces, etc., and even examples of use to enhance understanding.

Regarding where to show explanations, participants typically preferred separate elements (e.g., popups, sidebars), mainly because they did not want the explanations to be invasive and occupy space within the coding area. However, inline explanations may be accepted as comments close to the suggested code as long as their length is very limited. However, separate windows may allow developers to initiate an explanation process with a conversational agent, similar to what is possible to achieve with tools such as ChatGPT (OpenAI, 2024a).

Timing also plays a crucial role in showing the explanation. Some participants stated that explanations should be triggered manually using shortcuts, mouse clicks, or mouse hovering; notably, selecting code to explain with the mouse was a popular preference, also because it allows limiting the explanation to the context of the selection. Such an approach would have the advantage of reducing the amount of non-relevant explanation text to show to the user; moreover, it would also allow asking for further detailed explanations, limited to a specific piece of manually selected code, possibly to a chatbot. Some other participants instead expressed that brief explanations should be shown automatically (especially for novice developers), letting the user choose whether to receive more details on demand. Thus, all these findings indicate that adaptive, context-aware explanations can increase the understanding and trust of a tool dealing with complex or ambiguous code suggestions.

Participant preferences for delivering explanations are evidenced in Section 5.2, Table A.7 (7/8 groups favoring a separate window or pop-up), Table A.8 (6/8 groups preferring manual interaction), and Table A.9 (4/8 groups requesting minimal explanation with more details on request).

6.5. Personalization and design needs (RQ3, RQ4)

An aspect that emerged as essential to fit the CCT's behavior in different mental models is the explicit personalization of the CCT. There was, indeed, little consensus among participants towards one universally accepted solution; in fact, many individual factors, such as experience with coding, quality requirements, and personal preferences, play a role in determining the optimal behavior (and, therefore, acceptance) of the CCT. For example, developers want to customize when suggestions should appear, how to visualize them, and which granularity should be used by default. Choosing whether suggestions should also be suggested automatically resulted in being, indeed, crucial to users for the acceptance of the CCT, while they also expressed the preference to edit what manual action (e.g., shortcut, tog-

gle button, etc.) activates the code completion. In addition, customizing the suggestions' font, colors, and size and controlling the suggestions' position within the IDE also proved useful. Another critical point regarded the need to customize the granularity of suggestions to control the amount of code provided by the tool, both regarding the scope of the suggestion (function-level, control structure-level, etc.) and the number of provided alternatives.

The users expressed the need to customize aspects of generating explanations regarding the suggested code. Therefore, developers must be able to customize aspects such as the activation timing of explanations (whether to provide them proactively) and their granularity (such as the amount and frequency of comments within the generated code and the explanation abstraction).

Finally, customization of a CCT should include the choice of default stylistic code (such as coding conventions, variable names, and styles) when generating code suggestions, possibly adapting to other past projects. Therefore, the developer should be allowed to "fine-tune" the code-generating model with a selection of other code repositories to align the suggestions with a knowledge base representing the generated code's style. This customization should also allow the developer to set standards of quality and security to which the generated code has to comply.

Thus, a personalization layer in CCTs should be introduced to allow developers to define CCT settings to improve the tool's usability, acceptance, and trust. These customization options may aid in implementing a smoother running and adaptable CCT experience to strengthen the correspondence of the CCT conduct with developers' mental models.

7. Reifying the mental model: the ATHENA prototype

In this section, we put the mental model derived from our elicitation study into practice by presenting ATHENA, a prototype implementation that embodies and operationalizes the key design principles and the identified user interaction preferences.

ATHENA (AI Tool for Human-centered ENhanced coding Assistance) is a proof-of-concept CCT for Visual Studio Code IDE, developed to verify and demonstrate the feasibility of the actionable guidelines emerging from our study. The implementation incorporates high configurability, such as suggestion timing, granularity control, adaptive visualization (inline, sidebar, popup), and customizable explanation delivery that allows developers to tailor the extension to their preferences and workflows. As its namesake, Athena, the Greek goddess of skill and strategy, the tool aims to provide developers with precise, adaptable coding support.

The extension dynamically adapts to the user's coding preferences and environment, ensuring seamless integration into various workflows. The AI backend employs OpenAI's *GPT-4o mini*[1] to generate code suggestions and explanations. Of course, the AI model can be replaced with other similar solutions.

One of the major takeaways from the user study is that no "one-size-fits-all" approach exists to accommodate the needs of different types of users. Therefore, high customization was designated as a core feature in the tool's design. To this aim, *ATHENA* offers robust configurability through its *Settings Wizard Panel*, which is automatically displayed

[1] https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/

upon installation and remains accessible via a customizable shortcut. Settings are managed through Visual Studio Code's built-in infrastructure, supporting workspace and user-level configurations. The wizard is divided into two main sections: *General Settings* and *Shortcut Configuration*.

The general settings (Figure 5) enable users to customize the behavior and appearance of the CCT:

- *Suggestion Length*: A drop-down menu with options ranging from 1 to 10 adjusts the granularity of suggestions, balancing concise completions with detailed recommendations tailored to the user's needs.
- *Suggestion Appearance*: Users can choose from four display modes:
  - Inline: Embeds suggestions directly in the text editor for a seamless experience.
  - Tooltip: Shows suggestions in a hover menu near the cursor.
  - Side Window: Displays suggestions in a dedicated subpanel for enhanced focus.
  - Hybrid: Combines inline/tooltip suggestions for short completions and the side window for longer or more complex suggestions.
- *Suggestion Timing*: Users can toggle between:
  - Proactive: Automatically triggers suggestions as the user types, ideal for uninterrupted workflows.
  - Manual: Requires explicit activation through a shortcut, reducing distractions.
- *Source References*: A checkbox lets users include or omit sources of suggestions, ensuring transparency for users who value understanding the origin of generated code
- *Comments Frequency*: A slider allows users to adjust the frequency of comments in generated code, catering to both minimalist and documentation-focused coding styles.

The shortcut configuration section, on the other side, allows users to assign shortcuts to key actions:

- *Invoke chatbot*: Quickly access a persistent chatbot for coding queries, explanations, and guidance.
- *Open settings panel*: Revisit the settings at any time to adjust preferences.
- *Trigger manual suggestions*: Activate suggestions when working in manual mode.
- *Toggle auto-completion*: Enable or disable automated suggestions dynamically.

After the system is installed and customized, it can be used within Visual Studio Code. To assist developers during coding activities, ATHENA provides different interaction modalities. The first one is completely automatic and consists of proactive suggestions, which are automatically produced by ATHENA after a short idle time, and according to the preferences expressed by the users. If the developer wants to be more in control of the suggestions,

ATHENA provides two different solutions. In the first case, shortcuts can be triggered while writing pieces of code, while in the second case, a chatbot can be open to interactively receive suggestions.

ATHENA allows the developer to receive explanations for selected pieces of code within the editor via contextual IDE options. Specifically, a light-bulb icon appears at the top of the selected code, allowing for three distinct actions:

- *Explain the Code (Concise)*: Offers a concise summary of the code's functionality for the selected code.
- *Explain the Code (Detailed)*: Offers a deeper explanation of the purpose and structure of the selected code. This can also include the use of examples.
- *Open Chatbot*: Provides direct access to the chatbot with the selected code as context for further assistance.

Regardless of the solutions used by the developer to invoke the code suggestions and explanations, ATHENA's output can be visualized in two ways, according to the developer's preferences. The first solution consists of showing code/explanations within the code editor (either inline or in a tooltip). Another option is to show suggestions and explanations separately from the code editor within a side panel, as shown in Figure 6.

- *Suggestions/Completions*: Displays code completions only when the user has configured the display mode to "Side Window" or "Hybrid."
- *Explanations*: Provides detailed explanations for selected code, ensuring clarity and context for the developer's work.

Given the high degree of customizability, *ATHENA* addresses the issue of cold start by prompting the user with a question regarding their programming experience upon initial execution. Based on the user's response (which may be "beginner", "intermediate", or "advanced"), the tool adapts the default values in order to tailor its behavior. For instance, the results of the elicitation study suggest that activation should be on-demand for intermediate users, as this was the most preferred behavior overall. On the other hand, a novice user might benefit from a proactive approach that suggests code as they type. As another example, an intermediate user might benefit from the inclusion of documentation references in the suggestions, whereas an expert user might perceive them as superfluous or cumbersome. These default settings configurations were chosen arbitrarily; however, future work may inform the design of the optimal configurations for different categories of users. Nevertheless, after the initial question, the user is presented with the settings panel, where each configuration can be readily modified.

7.1. Use case scenarios

In the following, two scenarios are reported to clarify how the ATHENA tool can be installed and used the first time (Scenario 1, Section 7.1.1) and how the tool can be used by the same users in different contexts, thus demonstrating its suitability to different mental models of the same users (Scenario 2, Section 7.1.2). Two videos of the two scenarios are also attached to show how ATHENA works.

7.1.1. Scenario 1: ATHENA installation and first usage

Alex, a junior software developer at CodeCraft, is starting a new back-end API project for an e-commerce platform. To streamline their workflow, Alex installs the ATHENA Extension in Visual Studio Code. After installation, the extension prompts Alex with a popup asking their level of experience with programming. Alex answers the question by indicating "intermediate": this will automatically set the tool's parameters to an intermediate user profile, e.g., by disabling proactive suggestions. Nonetheless, the tool launches its *Settings Panel*, guiding Alex through the initial configuration process and giving them the possibility to change the default parameters. In the general settings, Alex sets the display mode to *Inline* to benefit from suggestions directly placed inside the editor. Alex thinks that suggestions that appear automatically while coding would be better and, therefore, chooses to set the suggestion trigger mode to *Proactive*. To fine-tune the experience, Alex adjusts the *suggestion length* slider to a medium level for balanced granularity. In the *Shortcut Configuration* section, Alex assigns shortcuts for invoking the chatbot, toggling auto-completion, and manually triggering suggestions. Satisfied with the setup, Alex saves the preferences and begins coding. As Alex starts writing the first few lines, inline suggestions appear seamlessly, offering useful completions for common coding patterns.

7.1.2. Scenario 2: adapting to debugging needs

A few weeks later, Alex is tasked with debugging a critical issue in the API's payment processing module. While the *Proactive Mode* had been helpful during development, Alex finds it distracting when troubleshooting errors. Recognizing the extension's flexibility, Alex opens the Settings Panel and switches the trigger mode to *Manual*, ensuring suggestions only appear when explicitly requested. Alex also reduces the *Suggestion Length* to receive concise completions that are better suited for debugging.

During debugging, Alex selects a block of code that seems problematic. A lightbulb icon appears next to the selected code, offering three options: "Explain the Code (Concise)", "Explain the Code (Detailed)", and "Open Chatbot". Alex selects the first option (*Explain the Code (Concise)*) and promptly receives a brief summary of the code's functionality. Alex wants to delve deeper into the details of the generated code; therefore, Alex selects the second option (*Explain the Code (Detailed)*), which provides a more in-depth explanation, to help the people understand the logic and potential pitfalls. To receive further guidance, Alex selects the *Open Chatbot* option; this invokes the chatbot, which provides context-aware suggestions for refactoring the code.

Later, Alex notices the Side Panel's explanation subpanel automatically updating with insights about the selected code. Combined with the concise suggestions displayed in the completion subpanel, Alex efficiently resolves the issue.

By tailoring the tool's behavior to their specific debugging needs, Alex maintains the focus and leverages the extension's features to complete the task effectively. The combination of personalized settings, code actions, and a context-aware chatbot reinforces the tool's versatility and value in diverse workflows.

8. Threats to validity

Even if our study provides useful insights about developers' mental models on CCTs, several limitations must be acknowledged.

External validity. The participants involved in our study were recruited among the students of two Italian universities, primarily third-year undergraduate computer science students. Although this sampling is appropriate for our exploratory goals, it does not represent the broader population of professional developers. Moreover, the sample skewed heavily male (48 out of 56), and few participants had extensive industry experience. In addition, students may lack the experience necessary to generate highly innovative or realistic design expectations. As a consequence, the generalizability of the findings to different developer populations is limited. Future research is needed to validate and extend these insights through studies involving participants from different industry settings and different skills, with more females, coming from multiple countries, and with more varied demographic backgrounds.

Internal validity. The workshop methodology adopted in our study, while useful for eliciting user mental models, can introduce biases. For instance, some participants may dominate the discussion while others may be quieter and less participatory. To mitigate this bias, we structured the workshops introducing an individual reflection phase for each question, and encouraged all participants to voice their opinions before group synthesis. Another potential problem regards the fact that researcher facilitation may still have unintentionally shaped some discussions. To mitigate this bias, audio recordings have been triangulated with observer transcription before performing the qualitative analysis.

Construct validity. The eight questions used in the workshops to guide the discussion were derived from the analysis of the existing literature on CCTs, with the aim of covering the critical interaction dimensions of CCTs. This structure supported a systematic coverage of relevant aspects; however, it may have constrained participants from reporting issues outside the predefined scope. To mitigate this problem, we introduced open-ended discussion and sketching tools that helped surface unanticipated questions.

Methodological limitations. Co-design workshops tend to foster breadth and richness of group discussion. However, participants, in particular students, may find it difficult to anticipate how they would interact with a real CCT, thus introducing a form of hypothetical bias. This is particularly critical for early-stage technologies, as in the case of our study. Furthermore, participants' comments may not fully reflect their behavior in the real world. As future work, to mitigate this problem, complementary methods should be adopted, for example, individual interviews, in-the-wild deployment, and usage logging.

9. Conclusion

The use of CCTs has seen a notable increase in recent years and chances are that they will be used more and more in the future. Nevertheless, their acceptance strongly depends on their alignment with developers' mental models. This study investigated the desiderata of CCTs through an elicitation study with programmers of varying expertise levels. Our findings indicate that, except for a few shared properties, there is no consensus on features such as activation

timing, position, and granularity of suggestions and explanations. This lack of agreement on the specifics of CCTs thus indicates that they necessitate a high degree of customizability.

Based on the findings of the elicitation study, we have also developed *ATHENA*, a prototype of a CCT for Visual Studio Code that is highly customizable and implements most of the features identified by our users. This tool is openly available to the research community and will be essential to conduct future user studies that assess how different characteristics of a CCT (e.g., the position of the code suggestion or the activation behavior) can affect its usability and user performance. In future work, we will investigate these differences with a controlled experiment to isolate different variables and measure their effects individually.

Limitations of this study mostly regard the convenience sampling methodology that was adopted, which led to a sample of very young participants, with limited to no expertise in companies. Therefore, despite the large sample size, the findings of this study are not directly generalizable to professional developers. To address this limitation we plan to conduct usablity studies with different categories of users. Such studies will be initially conducted in a laboratory setting, and then in-the-wild within an IT company; this will allow us to measure the effectiveness and usability of *ATHENA* compared to other popular CCTs. Moreover, we will observe how different features of a CCT can affect the usability and performance of users with varying degrees of expertise with different categories of users. Future work will also entail further developing our CCT and extending its functionalities by implementing the remaining features that emerged from the elicitation study. Moreover, we will explore the use of different LLMs as the AI code completion engine of the tool; in fact, to generate code suggestions and explanations *ATHENA* employs OpenAI's GPT-4o mini, an LLM which has a limited size but performs well on coding problems (OpenAI, 2024b). However, different models may prove more suitable for use in corporate settings. For instance, a non-commercial solution such as Meta's open-source LLM *LLama* (Meta, 2024) could be deployed on a company's server to prevent sharing confidential data, such as project repositories, with third parties. Furthermore, models of varying sizes could be contemplated in light of a tradeoff between costs and performance.

Acknowledgements

This research is partially funded by the Italian Ministry of University and Research (MUR) and by the European Union - NextGenerationEU, Mission 4, Component 2, Investment 1.1, under grant PRIN 2022 "DevProDev: Profiling Software Developers for Developer-Centered Recommender Systems" — CUP: H53D23003620006.

The research of Andrea Esposito is funded by a Ph.D. fellowship within the framework of the Italian "D.M. n. 352, April 9, 2022" - under the National Recovery and Resilience Plan, Mission 4, Component 2, Investment 3.3 - Ph.D. Project "Human-Centered Artificial Intelligence (HCAI) techniques for supporting end users interacting with AI systems", co-supported by "Eusoft S.r.l." (CUP H91I22000410007).

The research of Francesco Greco is funded by a PhD fellowship within the framework of the Italian "D.M. n. 352, April 9, 2022"- under the National Recovery and Resilience Plan, Mission 4, Component 2, Investment 3.3 - PhD

Project “Investigating XAI techniques to help user defend from phishing attacks”, co-supported by “Auriga S.p.A.” (CUP H91I22000410007).

Statements and Declaration

Conflicts of Interest

The authors declare no conflicts of interest.

CRediT authorship contribution statement

Conceptualization: Giuseppe Desolda; **Data curation:** Francesco Greco, Cesare Tucci; **Formal analysis:** Giuseppe Desolda, Francesco Greco, Cesare Tucci; **Funding acquisition:** Giuseppe Desolda; **Investigation:** Giuseppe Desolda, Andrea Esposito, Francesco Greco, Cesare Tucci; **Methodology:** Giuseppe Desolda, Andrea Esposito, Francesco Greco, Cesare Tucci; **Project administration:** Giuseppe Desolda; **Software:** Francesco Greco, Cesare Tucci; **Supervision:** Giuseppe Desolda; **Validation:** Giuseppe Desolda, Andrea Esposito; **Visualization:** Andrea Esposito; **Writing – original draft:** Giuseppe Desolda, Andrea Esposito, Francesco Greco, Cesare Tucci, Paolo Buono, Antonio Piccinno; **Writing – review & editing:** Giuseppe Desolda, Andrea Esposito, Francesco Greco, Cesare Tucci, Paolo Buono, Antonio Piccinno

References

T. Weber, M. Brandmaier, A. Schmidt, S. Mayer, Significant Productivity Gains through Programming with Large Language Models, Proc. ACM Hum.-Comput. Interact. 8 (2024) 1–29. URL: https://dl.acm.org/doi/10.1145/3661145. doi:10.1145/3661145.

Z. Cui, M. Demirer, S. Jaffe, L. Musolff, S. Peng, T. Salz, The Effects of Generative AI on High Skilled Work: Evidence from Three Field Experiments with Software Developers, 2024. URL: https://www.ssrn.com/abstract=4945566. doi:10.2139/ssrn.4945566.

S. Peng, E. Kalliamvakou, P. Cihon, M. Demirer, The Impact of AI on Developer Productivity: Evidence from GitHub Copilot, 2023. URL: https://arxiv.org/abs/2302.06590. doi:10.48550/ARXIV.2302.06590.

N. Sambasivan, R. Veeraraghavan, The Deskilling of Domain Expertise in AI Development, in: CHI Conference on Human Factors in Computing Systems, ACM, New Orleans LA USA, 2022, pp. 1–14. URL: https://dl.acm.org/doi/10.1145/3491102.3517578. doi:10.1145/3491102.3517578.

A. Sergeyuk, Y. Golubev, T. Bryksin, I. Ahmed, Using AI-based coding assistants in practice: State of affairs, perceptions, and ways forward, Information and Software Technology 178 (2025) 107610. URL: https://linkinghub.elsevier.com/retrieve/pii/S0950584924002155. doi:10.1016/j.infsof.2024.107610.

B. Shneiderman, Human-Centered AI, 1 ed., Oxford University Press, Oxford, 2022.

W. Xu, Toward Human-Centered Ai: A Perspective from Human-Computer Interaction, Interactions 26 (2019) 42–46. URL: https://dl.acm.org/doi/10.1145/3328485. doi:10.1145/3328485.

G. Desolda, A. Esposito, R. Lanzilotti, A. Piccinno, M. F. Costabile, From human-centred to symbiotic artificial intelligence: A focus on medical applications, Multimedia Tools and Applications In Press (2024). doi:10.1007/s11042-024-20414-5.

M. Izadi, J. Katzy, T. Van Dam, M. Otten, R. M. Popescu, A. Van Deursen, Language models for code completion: A practical evaluation, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, 2024, pp. 1–13.

R. A. Husein, H. Aburajouh, C. Catal, Large language models for code completion: A systematic literature review, Computer Standards & Interfaces (2024) 103917.

J. T. Liang, C. Yang, B. A. Myers, A large-scale survey on the usability of AI programming assistants: Successes and challenges, in: Proceedings of the 46th International Conference on Software Engineering (ICSE 2024), ACM, 2024, pp. 1–13. doi:10.1145/3597503.3608128.

C. Wang, J. Hu, C. Gao, Y. Jin, T. Xie, H. Huang, Z. Lei, Y. Deng, How Practitioners Expect Code Completion?, in: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ACM, San Francisco CA USA, 2023, pp. 1294–1306. URL: https://dl.acm.org/doi/10.1145/3611643.3616280. doi:10.1145/3611643.3616280.

P. Vaithilingam, E. L. Glassman, P. Groenwegen, S. Gulwani, A. Z. Henley, R. Malpani, D. Pugh, A. Radhakrishna, G. Soares, J. Wang, A. Yim, Towards more effective AI-assisted programming: A systematic design exploration to improve visual studio intellicode's user experience, in: Proceedings of the 45th International Conference on Software Engineering (ICSE) – Software Engineering in Practice Track (SEIP 2023), IEEE / ACM, 2023, pp. 185–195. doi:10.1109/ICSE-SEIP58684.2023.00022.

M. Mărășoiu, L. Church, A. F. Blackwell, An empirical investigation of code completion usage by professional software developers, in: Proceedings of the 26th Annual Workshop of the Psychology of Programming Interest Group (PPIG 2015), 2015, pp. 1–10.

M. Alenezi, M. Akour, AI-Driven Innovations in Software Engineering: A Review of Current Practices and Future Directions, Applied Sciences 15 (2025) 1344. URL: https://www.mdpi.com/2076-3417/15/3/1344. doi:10.3390/app15031344.

B. Martinović, R. Rozić, Impact of AI Tools on Software Development Code Quality, Springer Nature Switzerland, Cham, 2024, pp. 241–256. URL: https://link.springer.com/10.1007/978-3-031-62058-4_15. doi:10.1007/978-3-031-62058-4_15.

B. Shneiderman, Bridging the Gap Between Ethics and Practice: Guidelines for Reliable, Safe, and Trustworthy Human-centered AI Systems, ACM Transactions on Interactive Intelligent Systems 10 (2020) 1–31. URL: https://dl.acm.org/doi/10.1145/3419764. doi:10.1145/3419764.

National Transportation Safety Board, Collision Between a Car Operating with Automated Vehicle Control Systems and a Tractor-Semitrailer Truck Near Williston, Florida, May 7, 2016, Highway Accident Report NTSB/HAR-17/02, Washington, DC, 2017. URL: https://www.ntsb.gov/investigations/accidentreports/reports/har1702.pdf.

B. Shneiderman, Human-Centered Artificial Intelligence: Reliable, Safe & Trustworthy, International Journal of Human–Computer Interaction 36 (2020) 495–504. URL: https://www.tandfonline.com/doi/full/10.1080/10447318.2020.1741118. doi:10.1080/10447318.2020.1741118.

M. Cummings, Automation Bias in Intelligent Time Critical Decision Support Systems, in: AIAA 1st Intelligent Systems Technical Conference, American Institute of Aeronautics and Astronautics, Chicago, Illinois, 2004. URL: https://arc.aiaa.org/doi/10.2514/6.2004-6313. doi:10.2514/6.2004-6313.

A. Esposito, G. Desolda, R. Lanzilotti, The fine line between automation and augmentation in website usability evaluation, Scientific Reports 14 (2024) 10129. URL: https://www.nature.com/articles/s41598-024-59616-0. doi:10.1038/s41598-024-59616-0.

M. Mărășoiu, L. Church, A. Blackwell, An empirical investigation of code completion usage by professional software developers, in: Proceedings of the 26th Annual Workshop of the Psychology of Programming Interest Group, 2015.

J. Prather, B. N. Reeves, P. Denny, B. A. Becker, J. Leinonen, A. Luxton-Reilly, G. Powell, J. Finnie-Ansley, E. A. Santos, "It's Weird That it Knows What I Want": Usability and Interactions with Copilot for Novice Programmers, ACM Transactions on Computer-Human Interaction 31 (2024) 1–31. URL: https://dl.acm.org/doi/10.1145/3617367. doi:10.1145/3617367.

J. T. Liang, C. Yang, B. A. Myers, A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ACM, Lisbon Portugal, 2024, pp. 1–13. URL: https://dl.acm.org/doi/10.1145/3597503.3608128. doi:10.1145/3597503.3608128.

F. F. Xu, B. Vasilescu, G. Neubig, In-IDE Code Generation from Natural Language: Promise and Challenges, ACM Transactions on Software Engineering and Methodology 31 (2022) 1–47. URL: https://dl.acm.org/doi/10.1145/3487569. doi:10.1145/3487569.

C. Bird, D. Ford, T. Zimmermann, N. Forsgren, E. Kalliamvakou, T. Lowdermilk, I. Gazit, Taking Flight with Copilot: Early insights and opportunities of AI-powered pair-programming tools, Queue 20 (2022) 35–57. URL: https://dl.acm.org/doi/10.1145/3582083.

doi:10.1145/3582083.

B. Zhang, P. Liang, X. Zhou, A. Ahmad, M. Waseem, Demystifying Practices, Challenges and Expected Features of Using GitHub Copilot, International Journal of Software Engineering and Knowledge Engineering 33 (2023) 1653–1672. URL: https://www.worldscientific.com/doi/10.1142/S0218194023410048. doi:10.1142/S0218194023410048.

A. Sergeyuk, S. Titov, M. Izadi, In-IDE Human-AI Experience in the Era of Large Language Models; A Literature Review, in: 2024 IEEE/ACM First IDE Workshop (IDE), IEEE Computer Society, 2024, pp. 95–100. URL: https://www.computer.org/csdl/proceedings-article/ide/2024/058000a095/215yuQ6aI9i. doi:10.1145/3643796.3648463.

P. Vaithilingam, E. L. Glassman, P. Groenwegen, S. Gulwani, A. Z. Henley, R. Malpani, D. Pugh, A. Radhakrishna, G. Soares, J. Wang, A. Yim, Towards More Effective AI-Assisted Programming: A Systematic Design Exploration to Improve Visual Studio IntelliCode's User Experience, in: 2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), IEEE, Melbourne, Australia, 2023, pp. 185–195. URL: https://ieeexplore.ieee.org/document/10172834/. doi:10.1109/ICSE-SEIP58684.2023.00022.

D. A. Norman, Some Observations on Mental Models, in: Mental Models, Psychology Press, 1983.

D. A. Norman, The Design of Everyday Things, revised and expanded edition ed., The MIT Press, Cambridge, MA London, 2013.

T. Doi, Effects of asymmetry between design models and user models on subjective comprehension of user interface, Symmetry 13 (2021). URL: https://www.mdpi.com/2073-8994/13/5/795. doi:10.3390/sym13050795.

G. Desolda, A. Esposito, R. Lanzilotti, A. Piccinno, M. F. Costabile, From human-centered to symbiotic artificial intelligence: a focus on medical applications, Multimedia Tools and Applications (2024). URL: https://doi.org/10.1007/s11042-024-20414-5. doi:10.1007/s11042-024-20414-5.

J. A. Jacko (Ed.), The Human–Computer Interaction Handbook: Fundamentals, Evolving Technologies, and Emerging Applications, 3 ed., CRC Press, 2012. URL: https://www.taylorfrancis.com/books/9781439829448. doi:10.1201/b11963.

Y. Rogers, H. Sharp, J. Preece, Interaction Design: Beyond Human-Computer Interaction, 6 ed., John Wiley & Sons, Inc, Hoboken, 2023.

C. Lewis, Using the "Thinking-aloud" Method in Cognitive Interface Design, Research Report RC9265, IBM Thomas J. Watson Research Center, Yorktown Heights, NY, 1982. URL: https://dominoweb.draco.res.ibm.com/2513e349e05372cc852574ec0051eea4.html.

M. B. Rosson, J. M. Carroll, Scenario-Based Design, in: J. A. Jacko (Ed.), The Human–Computer Interaction Handbook: Fundamentals, Evolving Technologies, and Emerging Applications, 3 ed., CRC Press, 2012.

H. D. Lasswell, The structure and function of communication in society, in: W. Schramm, D. F. Roberts, W. Schramm (Eds.), The Process and Effects of Mass Communication, revised ed., University of Illinois Press, Urbana, Illinois, 1974, pp. 88–99.

G. Desolda, C. Ardito, M. Matera, Empowering End Users to Customize their Smart Environments: Model, Composition Paradigms, and Domain-Specific Tools, ACM Transactions on Computer-Human Interaction 24 (2017) 1–52. URL: https://dl.acm.org/doi/10.1145/3057859. doi:10.1145/3057859.

Z. Duric, W. Gray, R. Heishman, Fayin Li, A. Rosenfeld, M. Schoelles, C. Schunn, H. Wechsler, Integrating perceptual and cognitive modeling for adaptive and intelligent human-computer interaction, Proceedings of the IEEE 90 (2002) 1272–1289. URL: http://ieeexplore.ieee.org/document/1032808/. doi:10.1109/jproc.2002.801449.

A. Svyatkovskiy, S. K. Deng, S. Fu, N. Sundaresan, IntelliCode compose: Code generation using transformer, in: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ACM, Virtual Event USA, 2020, pp. 1433–1443. URL: https://dl.acm.org/doi/10.1145/3368089.3417058. doi:10.1145/3368089.3417058.

M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, W. Zaremba, Evaluating Large Language Models Trained on Code, 2021. URL: http://arxiv.org/abs/2107.03374. arXiv:2107.03374.

N. Friedman, Introducing GitHub Copilot: Your AI pair programmer, 2021. URL: https://github.blog/news-insights/product-news/introducing-github-copilot-ai-pair-programmer/.

S. Kim, J. Zhao, Y. Tian, S. Chandra, Code Prediction by Feeding Trees to Transformers, in: 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE), IEEE, Madrid, ES, 2021, pp. 150–162. URL: https://ieeexplore.ieee.org/document/9402114/. doi:10.1109/ICSE43902.2021.00026.

R. Li, L. B. allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. LI, J. Chim, Q. Liu, E. Zheltonozhskii, T. Y. Zhuo, T. Wang, O. Dehaene, J. Lamy-Poirier, J. Monteiro, N. Gontier, M.-H. Yee, L. K. Umapathi, J. Zhu, B. Lipkin, M. Oblokulov, Z. Wang, R. Murthy, J. T. Stillerman, S. S. Patel, D. Abulkhanov, M. Zocca, M. Dey, Z. Zhang, U. Bhattacharyya, W. Yu, S. Luccioni, P. Villegas, F. Zhdanov, T. Lee, N. Timor, J. Ding, C. S. Schlesinger, H. Schoelkopf, J. Ebert, T. Dao, M. Mishra, A. Gu, C. J. Anderson, B. Dolan-Gavitt, D. Contractor, S. Reddy, D. Fried, D. Bahdanau, Y. Jernite, C. M. Ferrandis, S. Hughes, T. Wolf, A. Guha, L. V. Werra, H. de Vries, StarCoder: May the source be with you!, Transactions on Machine Learning Research (2023). URL: https://openreview.net/forum?id=KoFOg41haE.

Q. Zheng, X. Xia, X. Zou, Y. Dong, S. Wang, Y. Xue, L. Shen, Z. Wang, A. Wang, Y. Li, T. Su, Z. Yang, J. Tang, CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Benchmarking on HumanEval-X, in: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, ACM, Long Beach CA USA, 2023, pp. 5673–5684. URL: https://dl.acm.org/doi/10.1145/3580305.3599790. doi:10.1145/3580305.3599790.

Amazon, What is CodeWhisperer?, 2024. URL: https://docs.aws.amazon.com/codewhisperer/latest/userguide/what-is-cwspr.html.

Codeium, Codeium | AI Code Completion & Chat, 2024. URL: https://codeium.com.

Replit Inc., Replit – Build software faster, 2024. URL: https://www.replit.com/.

Sourcegraph, Cody | AI coding assistant, 2025. URL: https://sourcegraph.com/cody.

Tabnine, Tabnine | AI code assistant, 2025. URL: https://www.tabnine.com/.

Apple Inc., Xcode 16 Release Notes, 2024. URL: https://developer.apple.com/documentation/xcode-release-notes/xcode-16-release-notes.

Blackbox AI, Chat Blackbox: AI Code Generation, Code Chat, Code Search, 2024. URL: https://www.blackbox.ai/.

N. Marquardt, K. Hinckley, S. Greenberg, Cross-device interaction via micro-mobility and f-formations, in: Proceedings of the 25th Annual ACM Symposium on User Interface Software and Technology, ACM, Cambridge Massachusetts USA, 2012, pp. 13–22. URL: https://dl.acm.org/doi/10.1145/2380116.2380121. doi:10.1145/2380116.2380121.

S. Voida, M. Podlaseck, R. Kjeldsen, C. Pinhanez, A study on the manipulation of 2D objects in a projector/camera-based augmented reality environment, in: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, ACM, Portland Oregon USA, 2005, pp. 611–620. URL: https://dl.acm.org/doi/10.1145/1054972.1055056. doi:10.1145/1054972.1055056.

V. Braun, V. Clarke, Using thematic analysis in psychology, Qualitative Research in Psychology 3 (2006) 77–101. URL: http://www.tandfonline.com/doi/abs/10.1191/1478088706qp063oa. doi:10.1191/1478088706qp063oa.

M. Asaduzzaman, C. K. Roy, K. A. Schneider, D. Hou, Cscc: Simple, efficient, context sensitive code completion, in: 2014 IEEE International Conference on Software Maintenance and Evolution, IEEE, 2014, pp. 71–80.

S. I. Ross, F. Martinez, S. Houde, M. Muller, J. D. Weisz, The Programmer's Assistant: Conversational Interaction with a Large Language Model for Software Development, in: Proceedings of the 28th International Conference on Intelligent User Interfaces, ACM, Sydney NSW Australia, 2023, pp. 491–514. URL: https://dl.acm.org/doi/10.1145/3581641.3584037. doi:10.1145/3581641.3584037.

T. Miller, Explanation in Artificial Intelligence: Insights from the Social Sciences, Artificial Intelligence 267 (2019) 1–38. URL: https://www.sciencedirect.com/science/article/pii/S0004370218305988. doi:10.1016/j.artint.2018.07.007.

A. Holzinger, A. Carrington, H. Müller, Measuring the Quality of Explanations: The System Causability Scale (SCS): Comparing Human and Machine Explanations, KI - Künstliche Intelligenz 34 (2020) 193–198. URL: http://link.springer.com/10.1007/s13218-020-00636-z. doi:10.1007/s13218-020-00636-z.

OpenAI, Chatgpt, 2024a. URL: https://chatgpt.com/.

OpenAI, Gpt-4o mini: advancing cost-efficient intelligence, 2024b. URL: https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/.

Meta, Llama 3.2: Revolutionizing edge ai and vision with open, customizable models, 2024. URL: https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/.

Appendix A. Themes and codes

Table A.2: Themes for Question 1

Q1. Where to show the code completion in the UI?			
Theme	***Theme frequency***	***Codes***	***Code frequency***
Separate window	**6**	Code 1: "Generated code should appear in a side window or tab" sideWindow: 1,3,4,5	4
		Code 2: "Generated code should appear in a tab at the bottom of the IDE" bottomWindow: 3	2
		Code 3: "Generated code should appear in a side movable window" movableTab: 7	2
Popup	**3**	Code 1: "Pop-up near the main code"	2
		Code 2: "Toggable pop-ups to view code completions"	1
Inline	**5**	Code 1: Inline code shown as a preview	2
		Code 2: Inline suggestion with a different font to differentiate it	1
		Code 3: Completion code should appear inline but be explictly accepted by the user	2
Adaptive visualization	**2**	Code 1: "Separate area for complex hints, inline for short ones"	2

Table A.3: Themes for Question 2

Q2. When should the suggestion be activated during the interaction?			
Theme	***Theme frequency***	***Codes***	***Code frequency***
Proactive	**4**	Code 1: "Trigger hint during typing automatically"	2
		Code 2: "Hints appear when user is blocked"	4
Manual	**5**	Code 1: "Manual trigger with shortcut"	2
		Code 2: "Manual activation preferred over proactive activation"	2
		Code 3: "Manual activation by selecting code"	2
		Code 4: "Manual activation option required"	3
		Code 5: "On-demand activation via comment"	1
Hybrid	**6**	Code 1: "Automatic hints during typing, manual by selection"	5
		Code 2: "Hybrid: automatic for obvious hints, manual otherwise"	1
		Code 3: "Manual hints for user when blocked"	1
When coding context is available	**4**	Code 1: "Hints appear after analyzing typing context"	4

Table A.4: Themes for Question 3

Q3. How should the suggested code be shown?			
Theme	***Theme frequency***	***Codes***	***Code frequency***
Inline	**5**	Code 1: "Change font to transparent for generated code"	3
		Code 2: Shorter suggested code is transparent	2
		Code 3: Inline suggestions for short code	2
		Code 4: "Color-coded suggestion types"	1
		Code 5: "Toggle color-coding for AI/human code"	1
Separate area	**7**	Code 1: "Separate window for detailed suggestions"	6
		Code 2: "Separate place for code suggestions"	1
		Code 3: Suggestion must be not invasive	2
Clicking an icon or badge	**2**	Code 1: "Pressable icon to signal suggestions availability"	2
Chatbot	**3**	Code 1: "The code suggestion should happen as a dialogue with a chatbot"	2
		Code 2: Chatbot for detailed requests	1

Table A.5: Themes for Question 4

Q4. What should be suggested?			
Theme	***Theme frequency***	***Codes***	***Code frequency***
Controlling granularity	**7**	Code 1: "Code should be given at varying granularity, choosable by the user"	2
		Code 2: "Code should be generated at a scope level defined by the user"	4
		Code 3: "Generated code should be limited to the function scope level"	1
		Code 4: "Generated code should be limited to the file scope level"	2
		Code 5: "Generated code should have a minimum amount of tokens"	1
		Code 6: "Generated code should at least be one line of code"	1
Different alternatives offered	**3**	Code 1: "Multiple code alternatives to choose from are preferred"	2
		Code 2: "In certain contexts, users should be able to choose from multiple code alternatives"	2
Minimal suggestions, details on-demand	**4**	Code 1: "Code should not be given right away, but be shown as an expandable preview"	1
		Code 2: "Suggested code should be given incrementally to foster the user's reasoning"	3
		Code 3: "Single suggestions (with no alternatives) are preferred"	1
		Code 4: "The code completion could be limited to just one token"	1
Codes and documentation	**2**	Code 1: "Show documentation together with the suggested code"	2
Code adapted to the context	**3**	Code 1: "The generated code should adapt to the context of the user (user profile and their work environment)"	3

Table A.6: Themes for Question 5

Q5. How should the suggestion be customized?			
Theme	***Theme frequency***	***Codes***	***Code frequency***
Activation timing of suggestions and explanations	**6**	Code 1: "Adjust activation timing manually"	2
		Code 2: "Enable/Disable automatic completion"	3
		Code 3: "Customize activation shortcut"	1
		Code 4: "Customize position of toggle button"	1
		Code 5: Activation of explanations should be customizable by the user	2
Appearance	**6**	Code 1: "Adjust font and style of suggestions"	4
		Code 2: "Define the position of the suggestions"	4
Coding style	**4**	Code 1: "Set the coding style for the generated code"	3
		Code 2: "Set the standards to comply with for the generated code"	3
		Code 3: "Define which data is used for fine tuning the code generation model"	2
Granularity of suggestions and explanations	**6**	Code 1: Customize the explanation granularity	2
		Code 2: "Control the presence of comments in the generated code"	2
		Code 3: "Control amount of code provided by tool"	4
		Code 4: "Number of different suggestions given"	1
		Code 5: "Limit the scope of the code generation"	2

Table A.7: Themes for Question 6

Q6. Where to show the code completion explanation in the UI?			
Theme	***Theme frequency***	***Codes***	***Code frequency***
Inline comments	**6**	Code 1: "Comment embedded in the code for context"	4
		Code 2: "Explanation be shown together with the generated code"	2
		Code 3: "Avoid to show complete explanations in comments but only show a brief title"	1
		Code 4: "Display explanation next to the code suggestion"	1
Separate window	**7**	Code 1: "Separate window or tab for explanations"	3
		Code 2: "Define the position of the suggestions"	1
		Code 3: Explanation must be shown separated from the code	1
		Code 4: "Display explanation in a dedicated lateral window"	4
		Code 5: "Show detailed explanation in a separated window"	1
		Code 6: "Provide explanation in a distinct popup"	2
		Code 7: "Sidebar tab to show explanations alongside code completion"	3
Chatbot	**3**	Code 1: "Explanations can be asked to a chatbot"	3

Table A.8: Themes for Question 7

Q7. When to show the explanation during the interaction?			
Theme	***Theme frequency***	***Codes***	***Code frequency***
Manually interacting with the suggested code	**6**	Code 1: "Explanations should be provided for pieces of code selected by the user"	3
		Code 2: "Explanations appear when clicking the mouse"	2
		Code 3: "Explanations appear when hovering over generated code"	2
		Code 4: "Explanations should be manually expandable"	3
Shortcut	**3**	Code 2: "Explanations are invoked by clicking a key or combination of keys"	3
Automatic short explanation, details on-demand	**3**	Code 1: "Automatic explanation for short suggestions"	3
		Code 2: "Explanation shown automatically to non-expert users"	1
		Code 3: "Long explanations should be provided only on demand"	1

Table A.9: Themes for Question 8

Q8. What should be explained?			
Theme	***Theme frequency***	***Codes***	***Code frequency***
Purpose	**5**	Code 1: "Explain why the code was generated in a specific manner"	4
		Code 2: "List pros and cons about the generated code"	1
		Code 3: "Explain different possible alternatives"	1
Examples and references	**5**	Code 1: "Provide real-world examples for better understanding"	4
		Code 2: "Add documentation references"	3
		Code 3: "Use trusted sources to report additional information in the explanation"	2
Contextual explanation	**4**	Code 1: "Provide suggestion of the selected piece of code"	4
Minimal explanation, details on-demand	**4**	Code 1: "Explanation should be minimal, with the possibility to manually ask for more detail"	2
		Code 2: "It would be useful to provide the explanation as a dialogue in a chat interaction"	1
		Code 3: "Provide an explanation preview that is expandable onDemand"	1
Functionality	**4**	Code 1: "Explanation should describe what the code is and does"	3
		Code 2: "Explanation should cover what data structures and code constructs appear in the generated code"	2
		Code 3: "Show pseudocode or flowcharts"	2
		Code 4: "Explanation should cover aspects that are not familiar to the user"	1
		Code 5: "Explanation should also explain low level concepts such as syntax and data types"	1

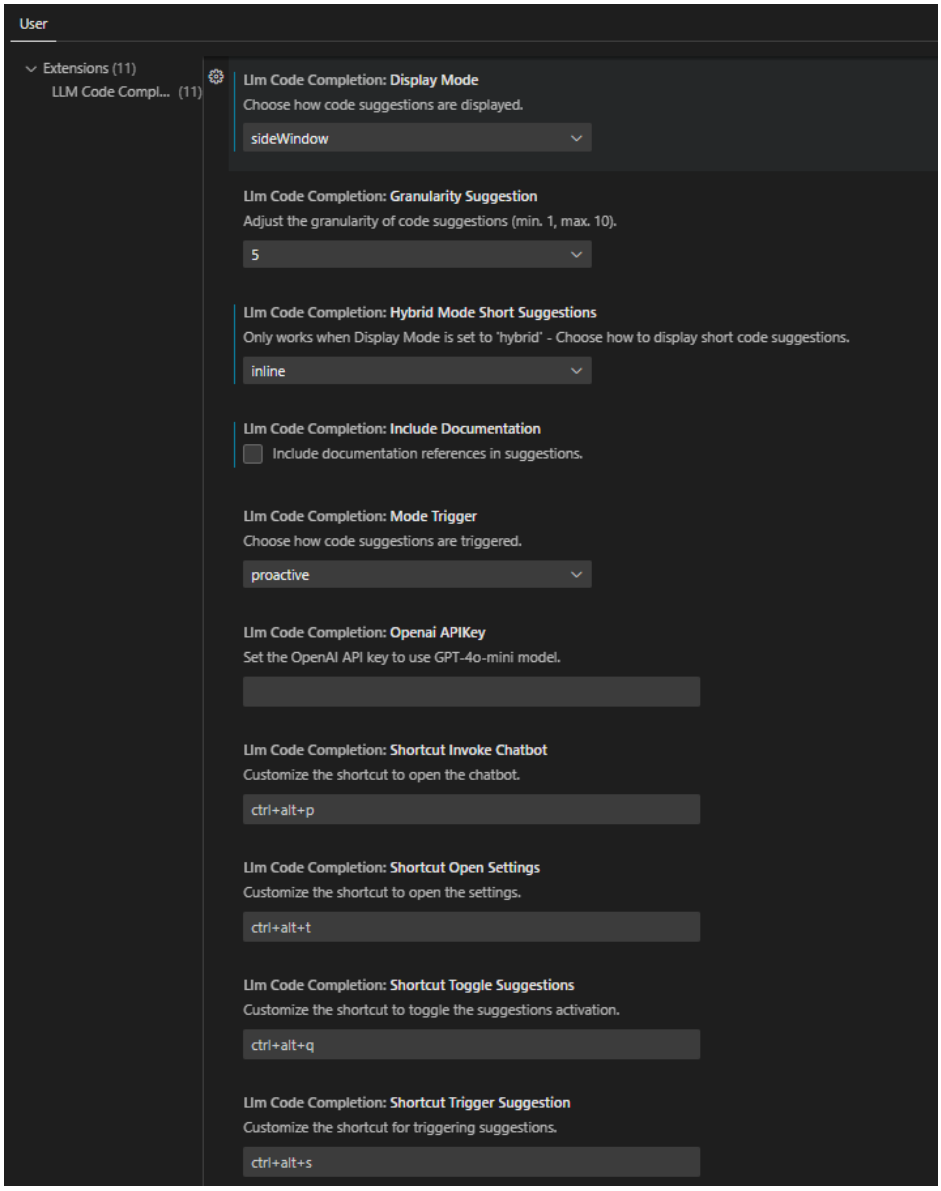

Figure 5: Screenshot of the settings provided with the prototype.

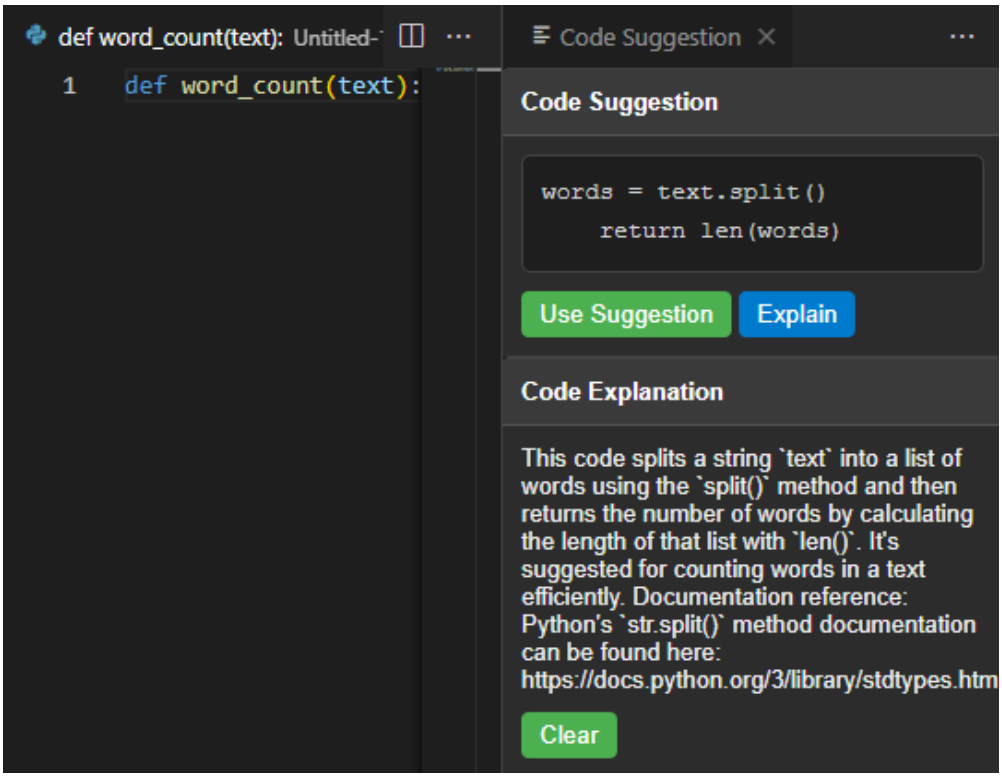

Figure 6: ATHENA interface: the user prefers to show suggestions in a separate panel.